PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

# FACE RECOGNITION ATTENDANCE SYSTEM

**AN INTERNSHIP REPORT**

***Submitted by,***


## SHIVPRASAD PATIL – 20221CSE0718


*Under the guidance of,*

## Mr. Jerrin Joe Francis

## BACHELOR OF TECHNOLOGY

**IN**

## COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

**PRESIDENCY UNIVERSITY**

**BENGALURU**

**APRIL 2026**

# PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

## PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

## BONAFIDE CERTIFICATE

Certified that this report **"Face Recognition Attendance System"** is a bonafide work of **Shivprasad Patil (20221CSE0718),** who has successfully carried out the internship work and submitted the report for partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

| | | |
|---|---|---|
| **Mr. Jerrin Joe Francis** | **Mr.Sakthi S** | 24/4/26 **Dr. Md Ziaur Rahaman** |
| Project Guide | Program Internship Coordinator | School Internship Coordinator |
| PSCS | PSCS | PSCS |
| Presidency University | Presidency University | Presidency University |

| | |
|---|---|
| **Dr.Asif Mohammed** | **Dr. Duraipandian N** |
| Head of the Department, | Dean |
| PSCS | PSCS / PSIS |
| Presidency University | Presidency University |

**Name and Signature of the Examiners**

| Sl.No | Name of the Examiner | Signature/Date |
|---|---|---|
| 1. | Mr. Senthil Kumar | 24/04 |
| 2. | Ms. Attar Mahay Sheetal | 24/4/26 |

## PRESIDENCY UNIVERSITY

## PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

### DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named SHIVPRASAD PATIL, hereby declare that the internship work titled **"FACE RECOGNITION ATTENDANCE SYSTEM"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

SHIVPRASAD PATIL USN: 20221CSE0718 Signature

PLACE:BENGALURU

DATE:24-April-2026

# INTERNSHIP COMPLETION CERTIFICATE

रेल पहिया कारखाना
**RAIL WHEEL FACTORY**
भारत सरकार / Govt. of India
रेल मंत्रालय / Ministry of Railways

महा प्रबंधक का कार्यालय,
कार्मिक विभाग
यलहंका, बैंगलूरु - 560 064
General Manager's Office,
Personnel Department
Yelahanka, Bangalore - 560 064

NO.RWF/NG-16/348/Pt III

DATE: 23.04.2026

## CERTIFICATE

This is to certify that **Mr. SHIVPRASAD PATIL, (20221CSE0718)**, B.Tech (Computer Science and Engineering) Student of Presidency University, has underwent Internship on "**Full Stack Web Development – AI Face Attendance System**" **at Rail Wheel Factory, Yelahanka, Bangalore** for a period of 03 months i.e., 06th January 2026 to 06th April 2026.

The student has shown keen interest in learning during the period he was with us for the Internship. We found him Enthusiastic, Diligent, Hard Working and Creative.

We wish a bright future and success in all his endeavors.

**(K.S.KIRAN)**
**SENIOR PERSONNEL OFFICER-I**

Tel : +91-80-28072402

Fax : +91-80-28460306

# ACKNOWLEDGEMENTS

For the successful completion of this internship work, I have received support and guidance from many individuals, to whom I express my deep sense of gratitude and indebtedness. I extend my sincere thanks to our respected Chancellor, Pro-Vice Chancellor, and Registrar for their constant encouragement and support throughout the internship.

I would like to express my heartfelt gratitude to my internal guide, **Mr. Jerrin Joe Francis**, Presidency School of Computer Science and Engineering, Presidency University, for his continuous guidance, motivation, and valuable suggestions during the course of this internship. His support played a vital role in the successful completion of this work.

I am also thankful to **Dr. Ashif Mohammed**, Professor and Head of the Department, Presidency School of Computer Science and Engineering, Presidency University, for his encouragement and mentorship.

I express my sincere thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, and **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering, along with the Management of Presidency University, for providing the necessary facilities and a supportive environment that contributed to the successful completion of my internship.

I am grateful to **Dr. Md Ziaur Rahaman**, Internship Coordinator, and **Mr. Sakthi S**, Program Internship Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance throughout the internship period.

I also extend my gratitude to the teaching and non-teaching staff of the Presidency School of Computer Science and Engineering, as well as staff from other departments, for their valuable assistance and cooperation.

**SHIVPRASAD PATIL**

# ABSTRACT

The internship project titled *"AI Face Recognition Attendance Management System"* focuses on automating traditional attendance methods using Artificial Intelligence and Computer Vision technologies. Conventional attendance systems are time-consuming, prone to human errors, and susceptible to proxy attendance, which affects reliability and efficiency. To address these challenges, this project proposes an automated, contactless solution that improves accuracy and reduces manual effort. The system is developed using React.js for the frontend, Flask for the backend, and advanced AI technologies such as InsightFace and OpenCV for face recognition. It captures real-time video through a webcam, detects faces, and identifies individuals by comparing facial features with stored datasets using deep learning-based embeddings.

The system ensures accurate and efficient attendance marking by automatically recording user identity along with date and time in a SQLite database. It also provides a web-based dashboard for managing users, viewing attendance records, and generating reports. The use of deep learning models enhances recognition accuracy even under varying lighting conditions, while OpenCV enables real-time processing. Performance testing shows that the system operates efficiently with minimal delay and high reliability. Overall, the project demonstrates a practical implementation of AI in automating attendance systems, improving productivity, security, and real-time monitoring capabilities.

# TABLE OF CONTENTS


| Sl. No. | Title | Page No. |
|---|---|---|
| | Declaration | iii |
| | Acknowledgement | v |
| | Abstract | vi |
| | List of Figures | ix |
| | List of Tables | x |
| | Abbreviations | xi |
| 1. | Introduction<br>1.1 Background<br>1.2 Statistics of Internship<br>1.3 Prior existing technologies<br>1.4 Proposed approach<br>1.5 Objectives<br>1.6 SDGs<br>1.7 Overview of internship report | 1-6 |
| 2. | Literature Review<br>2.1 Introduction to Attendance Systems<br>2.2 traditional Attendance Systems<br>2.3 Face Recognition Technology<br>2.4 AI – Based Face Recognition Systems<br>2.5 Computer Vision Attendance Systems<br>2.6 Deep learning In Face Recognition<br>2.7 Existing Face Recognition Systems<br>2.8 Technologies Used in Modern System<br>2.9 Advantages of AI-Based Attendance Systems<br>2.10 Summary | 7-12 |
| 3. | Methodology<br>3.1 Development Approach<br>3.2 Requirement Analysis<br>3.3 System Design<br>3.4 Frontend Development | |

| | | |
|---|---|---|
| | 3.5 Backend Development<br>3.6 AI Integration<br>3.7 Implementation Phases<br>3.8 Testing Methodology<br>3.9 Deployment<br>3.10 Security Measures<br>3.11 Performance Optimization<br>3.12 Advantages Of The Methodology | 13-20 |
| 4. | Project Management<br>4.1 Internship timeline<br>4.2 Risk analysis<br>4.3 Internship budget | 21-25 |
| 5. | Analysis and Design<br>5.1 Requirements<br>5.2 Block Diagram<br>5.3 System Flow Chart<br>5.4 Choosing devices<br>5.5 Designing units<br>5.6 Standards<br>5.7 Mapping with TWF reference model layers<br>5.8 Domain model specification<br>5.9 Communication model<br>5.10 Deployment level<br>5.11 Functional view<br>5.12 Mapping IoT deployment level with functional view<br>5.13 Operational view<br>5.14 Other Design | 26-38 |
| 6. | Hardware, Software and Simulation<br>6.1 Hardware<br>6.2 Software development tools<br>6.3 Software code<br>6.4 Simulation | 39-48 |
| 7. | Evaluation and Results | |

| | | |
|---|---|---|
| | 7.1 Test points<br>7.2 Test plan<br>7.3 Test result<br>7.4 Insights | 49-55 |
| 8. | Social, Legal, Ethical, Sustainability and Safety Aspects<br>8.1 Social aspects<br>8.2 Legal aspects<br>8.3 Ethical aspects<br>8.4 Sustainability aspects<br>8.5 Safety aspects | 56-58 |
| 9. | Conclusion | 59 |
| | References | 60 |
| | Appendix | 61-66 |

# LIST OF FIGURES


| Sl. No. | Figure Name | Caption | Page No. |
|---|---|---|---|
| 1 | Figure 1.6 | SDGs (Sustainable Development Goals) | 06 |
| 2 | Figure 4.1 | Internship TimeLine Visualization | 22 |
| 3 | Figure 5.2 | Block Diagram | 27 |
| 4 | Figure 5.3 | System Flow Chart | 28 |
| 5 | Figure 5.6 | MVC Architecture | 30 |
| 6 | Figure 5.8 | Domine Model Specification | 31 |
| 7 | Figure 5.11 | Functional View | 33 |
| 8 | Figure 5.13 | Operational View | 35 |
| 9 | Figure 5.13.1 | API Key Working | 36 |
| 10 | Figure 6.4.1 | Admin Login | 44 |
| 11 | Figure 6.4.2 | Dashboard Page | 44 |
| 12 | Figure 6.4.3 | Attendance Page | 45 |
| 13 | Figure 6.4.4 | Gallery Page | 45 |
| 14 | Figure 6.4.5 | Person Page | 46 |
| 15 | Figure 6.4.6 | Report Page | 46 |
| 16 | Figure 7.1 | Test Points | 47 |
| 17 | Figure 7.2 | Test Plan | 50 |

# LIST OF TABLES


| Sl. No. | Table Name | Table Caption | Page No. |
|---|---|---|---|
| 1 | Table 4.3 | Budget Details | 25 |

# ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| API | Application Programming Interface |

# Chapter1

# INTRODUCTION

The AI Face Recognition Attendance Management System is an advanced solution designed to automate and improve the traditional attendance process using Artificial Intelligence (AI) and Computer Vision technologies. In many organizations such as colleges, schools, and industries, attendance is still recorded manually or using basic biometric systems. These traditional methods are often time-consuming, prone to human errors, and vulnerable to proxy attendance.

With the rapid advancement of technology, there is a growing need for intelligent systems that can automate routine tasks efficiently. Face recognition is one of the most powerful applications of AI, which enables the identification of individuals based on their unique facial features. This project leverages face recognition technology to build a smart attendance system that automatically marks attendance without human intervention.

The system captures live video through a webcam, detects human faces using OpenCV, and recognizes them using deep learning models provided by InsightFace. Once a face is recognized, the system records attendance in the database with accurate date and time. This eliminates the need for manual entry and significantly reduces the chances of errors or manipulation.

The application is developed using modern web technologies. The frontend is built using React.js, which provides an interactive and user-friendly interface. The backend is implemented using Flask, which handles API requests, data processing, and communication between modules. The system follows a client-server architecture, ensuring scalability and efficient data flow.

This project was developed during the internship at Rail Wheel Factory (RWF), Bengaluru, where the focus was on applying AI and full-stack development skills to solve real-world problems. The system is designed to be efficient, accurate, and easy to use, making it suitable for deployment in various environments.

## 1.1 Background

Attendance management is an essential activity in educational institutions and workplaces. Traditionally, attendance is recorded manually using registers or through basic systems such as ID cards or biometric devices. However, these methods have several drawbacks:

Manual attendance systems are time-consuming and often disrupt the workflow of classes or meetings. They also have a high probability of human errors, such as incorrect entries or missed records. Additionally, these systems are highly susceptible to proxy attendance, where individuals can mark attendance on behalf of others. Maintaining large volumes of attendance data manually becomes difficult over time and requires additional manpower.

With the emergence of Artificial Intelligence, face recognition has become a reliable and efficient method for identification. AI-based systems can automatically detect and recognize individuals, making them suitable for attendance management.

This project utilizes AI and Computer Vision to overcome the limitations of traditional attendance systems and provide a modern, automated solution.

## 1.2 Statistics of Internship

The internship provided valuable exposure to real-world application development and the integration of Artificial Intelligence technologies. The project was carried out at Rail Wheel Factory (RWF), Bengaluru, over a duration of approximately six to eight weeks. The primary domain of the internship was Artificial Intelligence and Full Stack Development.

During the internship, various technologies were utilized, including React.js for frontend development, Flask for backend processing, OpenCV for image processing, InsightFace for deep learning-based face recognition, and SQLite for database management. The project was divided into multiple modules, such as face registration, dataset collection, model training, live face recognition, and attendance logging.

This hands-on experience helped in understanding the complete development lifecycle of a real-time system, from requirement analysis to deployment. It also enhanced technical skills in AI integration, web development, and system design.

The internship helped in gaining hands-on experience in designing, developing, and deploying a real-time system.

## 1.3 Prior Existing Technologies

Before the development of AI-based attendance systems, several traditional methods were used:

**1. Manual Attendance System**

The manual attendance system is one of the most traditional methods used for recording attendance, where data is maintained using registers or paper-based records. Although simple in implementation, this approach is highly time-consuming and prone to human errors such as incorrect entries or missed records. Additionally, maintaining and organizing large volumes of attendance data becomes difficult over time, making the system inefficient and unreliable for modern organizational needs.

**2. RFID-Based Systems**

RFID-based attendance systems utilize identification cards to record attendance, allowing users to mark their presence by scanning or tapping their cards. While this method is faster and more convenient than manual systems, it has certain limitations in terms of security and reliability. One of the major drawbacks is that these systems can be easily misused, as individuals may share their ID cards with others, leading to proxy attendance and inaccurate records.

**3. Biometric Systems**

Biometric attendance systems rely on unique physiological characteristics such as fingerprints or iris patterns to identify individuals and record attendance. These systems provide a higher level of security compared to traditional methods, as biometric traits are difficult to replicate. However, the implementation of such systems requires specialized and often expensive hardware, increasing the overall cost. Additionally, touch-based biometric devices, such as fingerprint scanners, may raise hygiene concerns in shared environments, especially when used by a large number of individuals.

**Limitations of Existing Systems:**

Existing attendance systems suffer from several limitations that impact their efficiency and reliability. One of the major drawbacks is the lack of automation, which results in increased manual effort and time consumption. Additionally, these systems are vulnerable to fraudulent practices such as proxy attendance, where unauthorized individuals can mark attendance on behalf of others. The overall maintenance cost is also relatively high, especially for systems requiring dedicated hardware and regular upkeep. Furthermore, many traditional solutions have limited scalability, making them unsuitable for large organizations with a high number of users.

These limitations highlight the need for an intelligent and automated system.

## 1.4 Proposed Approach

The proposed system is an AI-based Face Recognition Attendance System that automates the attendance process.

**Working Approach:**

1. Capture live video using webcam
2. Detect faces using OpenCV
3. Extract facial features using InsightFace
4. Compare with stored dataset
5. Identify the person
6. Mark attendance automatically

**Advantages:**

The proposed AI-based attendance system offers several significant advantages over traditional methods. It is a fully automated solution that eliminates the need for manual intervention, thereby improving efficiency and reducing operational effort. The system

ensures high accuracy by utilizing advanced face recognition algorithms, minimizing errors in attendance recording. Additionally, it supports real-time processing, allowing attendance to be marked instantly as individuals are recognized. Since the system operates without physical contact, it provides a hygienic and user-friendly experience. Furthermore, it is designed to be secure and reliable, ensuring that attendance data is accurately recorded and protected from unauthorized access.

## 1.5 Objectives

The primary objective of this project is to develop an AI-based attendance management system that enhances the efficiency and reliability of attendance recording. The system aims to automate the entire attendance process, thereby reducing manual effort and minimizing human errors associated with traditional methods. Another important objective is to prevent proxy attendance by ensuring that only authorized individuals are identified through face recognition technology. The project also focuses on providing real-time attendance tracking, enabling accurate and instant recording of attendance data. In addition, the system is designed to offer a user-friendly web interface that simplifies interaction for users and administrators. Overall, the project seeks to improve operational efficiency, accuracy, and productivity through the integration of modern technologies.

## 1.6 SDGs (Sustainable Development Goals)

The proposed AI-based Face Recognition Attendance System aligns with the United Nations Sustainable Development Goals (SDGs), specifically SDG 9: Industry, Innovation and Infrastructure and SDG 4: Quality Education. SDG 9 is addressed through the integration of advanced technologies such as Artificial Intelligence and Computer Vision, which promote innovation and the development of smart digital infrastructure. Additionally, the system supports SDG 4: Quality Education by improving attendance management in educational institutions, ensuring accuracy, transparency, and efficient monitoring of student participation. By reducing manual processes and enabling automation, the project contributes to the adoption of modern, technology-driven solutions in both academic and organizational environments.

1.6 SDGs (Sustainable Development Goals)

The proposed AI-based Face Recognition Attendance System aligns with the United Nations Sustainable Development Goals (SDGs), particularly SDG 9: Industry, Innovation and Infrastructure and SDG 4: Quality Education. The project contributes to these goals by integrating modern technologies and improving efficiency in attendance management systems.

SDG 9: Industry, Innovation and Infrastructure

The system promotes innovation by utilizing advanced technologies such as Artificial Intelligence, Computer Vision, and deep learning models. It supports the development of smart digital infrastructure by replacing traditional manual systems with automated solutions. The implementation of real-time face recognition enhances technological adoption in organizations and encourages digital transformation. Additionally, the use of scalable web technologies ensures that the system can be expanded and adapted for large-scale industrial and institutional applications.

SDG 4: Quality Education

The system contributes to quality education by improving attendance tracking in educational institutions. It ensures accurate and transparent recording of student attendance, which helps in better monitoring of academic participation. By eliminating proxy attendance, the system promotes discipline and accountability among students. Furthermore, the automation of attendance processes allows educators to focus more on teaching rather than administrative tasks, thereby enhancing the overall learning environment.

Additional Contribution: Efficiency and Productivity

The system improves operational efficiency by reducing manual effort and minimizing errors in attendance recording. Automated processes save time and resources, allowing institutions and organizations to operate more effectively. This contributes to increased productivity and better management of human resources.

Additional Contribution: Digital Transformation and Smart Systems

The project supports the transition towards smart systems by integrating AI-driven solutions into everyday processes. It encourages the adoption of digital technologies in institutions and workplaces, promoting a more modern, efficient, and technology-driven environment. This aligns with global efforts to build sustainable and intelligent systems for future development.

**The project aligns with the following United Nations Sustainable Development Goals (SDGs):**

Fig : 1.6 SDGs (Sustainable Development Goals)

## 1.7 Overview of Internship Report

This report is structured into multiple chapters:

- **Chapter 1:** Introduction
- **Chapter 2:** Literature Survey
- **Chapter 3:** Methodology
- **Chapter 4:** Project Management
- **Chapter 5:** Analysis and Design
- **Chapter 6:** Implementation
- **Chapter 7:** Evaluation and Results
- **Chapter 8:** Social and Ethical Aspects
- **Conclusion**
- **References & Appendix**

This structured report provides a comprehensive understanding of the project and the work carried out during the internship.

# Chapter 2

## LITERATURE SURVEY

The literature survey provides an overview of existing technologies, research work, and systems related to attendance management and face recognition. It helps in understanding the evolution of attendance systems, their limitations, and the need for advanced AI-based solutions. This chapter focuses on traditional attendance systems, biometric technologies, and modern AI-based face recognition systems.

### 2.1 Introduction to Attendance Systems

Attendance management systems play a crucial role in recording and monitoring the presence of individuals in organizations such as educational institutions and workplaces [1]. Over time, these systems have evolved from traditional manual methods to automated digital solutions. Initially, attendance was recorded using paper-based registers, which were time-consuming and prone to errors. With technological advancements, systems such as RFID and biometric attendance were introduced to improve efficiency and accuracy [2]. However, these systems still faced challenges related to cost, scalability, and security. In recent years, the emergence of Artificial Intelligence and Computer Vision has enabled the development of smart attendance systems that automatically recognize individuals using facial features, providing a more efficient and reliable solution [3].

### 2.2 Traditional Attendance Systems

#### 2.2.1 Manual Attendance System

The manual attendance system is one of the earliest methods used for recording attendance [1]. It involves maintaining physical registers where attendance is marked manually. While this method is simple and does not require technical knowledge, it is highly time-consuming and prone to human errors. Additionally, it allows proxy attendance and becomes difficult to manage as the volume of data increases.

#### 2.2.2 RFID-Based Systems

RFID (Radio Frequency Identification) systems were introduced to automate attendance recording and improve efficiency [2]. These systems use ID cards that can be scanned to mark attendance quickly. Although they are faster than manual systems, they still suffer from issues such as proxy attendance, as cards can be shared. Moreover, RFID systems require additional hardware, increasing the overall cost and maintenance.

### 2.2.3 Biometric Systems

Biometric systems use unique biological characteristics such as fingerprints or iris patterns for identification [4]. These systems provide better security and reduce proxy attendance compared to RFID systems. However, they require expensive hardware and may raise hygiene concerns, especially in touch-based systems. Additionally, they can be slow when handling large groups.

## 2.3 Face Recognition Technology

Face recognition is a biometric technology that identifies individuals based on facial features [3]. It involves face detection, feature extraction, and face matching. Machine learning and deep learning algorithms are used to improve accuracy and efficiency in recognizing individuals.

## 2.4 AI-Based Face Recognition Systems

With the advancement of Artificial Intelligence, face recognition systems have become more accurate and efficient [5]. These systems use deep learning techniques to analyze facial features and perform real-time recognition. They also provide contactless operation, making them more hygienic and user-friendly.

## 2.5 Computer Vision in Attendance Systems

Computer Vision is a field of Artificial Intelligence that enables machines to interpret visual data from images and videos [6]. In attendance systems, it is used for capturing images, detecting faces, tracking movement, and recognizing individuals. Libraries such as OpenCV are widely used for implementing these functionalities.

## 2.6 Deep Learning in Face Recognition

Deep learning techniques significantly improve the accuracy of face recognition systems [5]. Convolutional Neural Networks (CNNs) are used to extract features from images, while face

embeddings convert these features into numerical representations. These embeddings are then compared using feature matching techniques for accurate recognition.

### 2.7 Existing Face Recognition Attendance Systems

Existing face recognition attendance systems use technologies such as OpenCV and machine learning models to automate attendance [6]. These systems store data in databases and provide web-based dashboards for monitoring. However, they face challenges such as reduced accuracy in poor lighting conditions, high computational requirements, and complex implementation.

### 2.8 Technologies Used in Modern Systems

Modern attendance systems are developed using a combination of technologies such as React.js for frontend, Flask or Node.js for backend, and SQLite or MongoDB for database management [7]. AI technologies such as OpenCV and InsightFace are used for face detection and recognition, ensuring high accuracy and performance.

### 2.9 Advantages of AI-Based Attendance Systems

AI-based attendance systems offer several advantages, including automation, high accuracy, and real-time processing [3]. They eliminate proxy attendance, improve efficiency, and provide secure and reliable attendance tracking without physical contact.

### 2.10 Summary

The literature survey highlights the evolution of attendance systems from manual methods to advanced AI-based solutions. Traditional systems have limitations such as inefficiency and lack of security, while AI-based systems provide automation, accuracy, and scalability [3]. The proposed system builds upon these technologies to create an efficient and reliable attendance management solution.

### 2.11 Comparative Analysis of Existing Systems

A comparative analysis of different attendance systems highlights the strengths and limitations of each approach. Traditional manual systems are simple and cost-effective but suffer from inefficiency, human errors, and lack of scalability. RFID-based systems improve speed and usability; however, they are vulnerable to proxy attendance due to card sharing and require

additional hardware. Biometric systems, such as fingerprint and iris recognition, offer higher security and uniqueness but involve high implementation costs and may raise hygiene concerns in shared environments.

In contrast, AI-based face recognition systems provide a contactless, automated, and highly efficient solution. These systems leverage deep learning techniques to accurately identify individuals in real time. They eliminate the need for physical interaction and significantly reduce the chances of fraud. However, they may face challenges such as dependency on lighting conditions, camera quality, and computational requirements. Despite these limitations, AI-based systems offer a more scalable and modern approach compared to traditional methods.

### 2.12 Research Gap

Despite the advancements in attendance management systems, several gaps still exist in current implementations. Many existing systems struggle with maintaining high accuracy under varying environmental conditions such as poor lighting or occlusions. Additionally, some systems require high computational resources, making them less suitable for low-cost environments. Real-time performance is another challenge, as delays in recognition can affect system efficiency. Furthermore, many solutions lack user-friendly interfaces and seamless integration between frontend, backend, and AI components.

The proposed system addresses these gaps by integrating efficient deep learning models with optimized processing techniques to achieve reliable real-time performance. It focuses on creating a user-friendly web interface, reducing computational overhead, and ensuring secure and accurate attendance tracking. By combining AI, web technologies, and database management, the system provides a balanced solution that improves usability, scalability, and overall system performance.

### 2.13 Conclusion of Literature Review (Optional Add-on)

The literature review demonstrates that while traditional and semi-automated attendance systems have contributed to improving efficiency, they still suffer from various limitations related to accuracy, security, and scalability. AI-based face recognition systems emerge as a powerful alternative by offering automation, real-time processing, and improved reliability. The proposed system builds upon these advancements and aims to overcome existing challenges by providing a robust, efficient, and user-friendly attendance management solution.

**Table: Summary of Literature Review**

| System Type | Technology Used | Advantages | | Limitations |
|---|---|---|---|---|
| Manual Attendance | Paper Registers | Simple, low cost, easy to implement | | Time-consuming, human errors, proxy attendance |
| RFID-Based System | ID Cards, RFID Reader | Fast, easy to use | | Card sharing (fraud), hardware cost, card loss |
| Biometric System | Fingerprint/Iris | High security, unique identification | | Expensive, hygiene issues, slow for large groups |
| Face Recognition | Computer Vision, ML | Contactless, automated, accurate | | Affected by lighting, requires good dataset |
| AI-Based Systems | Deep Learning, CNN | High accuracy, real-time processing | | High computation, complex implementation |
| Existing Systems | OpenCV, ML Models | Automated, dashboard-based monitoring | | Limited accuracy, performance issues |
| Proposed System | AI + Web Tech (React, Flask, InsightFace) | Fully automated, secure, real-time, scalable | | Depends on lighting & hardware quality |

**Chapter 3**

**Methodology**

The methodology describes the systematic approach followed during the development of the AI Face Recognition Attendance Management System. It includes planning, requirement analysis, system design, implementation, testing, and deployment. The methodology ensures that the project is developed in a structured, efficient, and reliable manner.

### 3.1. Development Approach

The project follows the Agile Development Methodology, which focuses on iterative development and continuous improvement.

**Key Features of Agile:**

The Agile development methodology offers several key features that enhance the efficiency and quality of software development. It emphasizes development in small, manageable modules, allowing incremental progress and better control over the project. Continuous testing and feedback are integral to Agile, enabling early detection of issues and timely improvements. This approach also facilitates easy debugging and refinement of individual components. Furthermore, Agile is highly flexible and adaptive, allowing the system to evolve based on changing requirements and feedback throughout the development lifecycle.

Each phase of the project was completed step-by-step, ensuring proper functionality before moving to the next phase.

## 3.2. REQUIREMENT ANALYSIS

Requirement analysis involves identifying the needs and functionalities of the system.

### 3.2.1 Functional Requirements

The functional requirements of the system define the core operations that must be performed

to ensure proper functionality. The system should support user registration, allowing new individuals to be added to the database. It must enable face dataset capture using a camera, followed by face encoding and model training to prepare the recognition system. Additionally, the system should perform real-time face recognition to identify individuals accurately and automatically mark attendance. It should also provide the capability to view attendance records and include a dashboard interface for efficient management and monitoring of users and attendance data.

### 3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards of the system. The system should ensure high accuracy in face recognition to maintain reliable attendance records. It must provide fast processing and quick response times to support real-time operations. Secure data handling is essential to protect sensitive user information and prevent unauthorized access. Additionally, the system should offer a user-friendly interface to enhance usability and ensure smooth interaction for users and administrators. The design should also be scalable, allowing the system to efficiently handle an increasing number of users without affecting performance.

## 3.3. SYSTEM DESIGN

The system follows a Client-Server Architecture**.**

**Components:**

The system is composed of several key components that work together to ensure efficient functionality. The frontend, developed using React.js, provides the user interface through which users interact with the system. The backend, implemented using a Flask API, handles processing, business logic, and communication between different modules. The AI module, which integrates InsightFace and OpenCV, is responsible for performing face detection and recognition tasks. Additionally, a SQLite database is used for data storage, maintaining user information and attendance records in an organized and efficient manner.

Architecture Flow (You can draw diagram)

User → React Frontend → Flask Backend → AI Model → Database

## 3.4 FRONTEND DEVELOPMENT

**AI Face Attendance System**

The frontend is developed using React.js**,** which enables the creation of dynamic and interactive user interfaces for the attendance system.

**Key features of frontend development:**

The frontend of the system is designed with several key features to ensure an efficient and user-friendly experience. It follows a component-based architecture, which promotes modular development and easier maintenance of the application. The system includes form handling mechanisms for user registration, allowing new individuals to be added seamlessly. A live camera interface is integrated to capture facial data for recognition purposes. Additionally, the frontend provides real-time attendance display, enabling users to view attendance updates instantly. The interface is also designed to be responsive, ensuring smooth performance across different devices such as desktops, tablets, and mobile screens.

The user interface is designed to be simple and user-friendly, allowing users to easily register, capture face data, and view attendance records.

## 3.5 BACKEND DEVELOPMENT

The backend is implemented using Python Flask**,** which handles server-side operations and integrates AI functionalities.

**Main functions of the backend:**

The backend of the system is responsible for handling core operations and ensuring seamless functionality across different components. Its main functions include processing API requests, managing user data and attendance records, and executing the face recognition logic. It also integrates AI models such as InsightFace and OpenCV to enable accurate face detection and recognition. Communication between the frontend and backend is facilitated through RESTful APIs, which ensure smooth, efficient, and reliable data transfer between system components.

RESTful APIs are used for communication between frontend and backend, ensuring smooth and efficient data transfer between system components.

## 3.6 AI INTEGRATION

Artificial Intelligence is a key component of the project. It plays an important role in enabling accurate and automated face recognition for attendance management.

The system uses advanced AI models to detect and recognize human faces in real-time using computer vision techniques.

**AI functionalities include:**

The AI functionalities form the core of the system, enabling automated and accurate attendance management. The system performs face detection from live video streams using OpenCV, identifying human faces in real time. It then applies deep learning models from InsightFace to encode facial features into unique numerical representations. These embeddings are compared with the stored dataset to recognize individuals accurately. Once a match is identified, the system automatically detects registered users and marks their attendance. All these processes are executed in real time, ensuring fast and efficient attendance recording without manual intervention.

The AI module captures live video input from the webcam, processes each frame to detect faces, and extracts facial features. These features are compared with the trained dataset to identify the person.

Once a match is found, the system automatically marks attendance with date and time in the database. This reduces manual effort, improves accuracy, and prevents proxy attendance.

## 3.7 IMPLEMENTATION PHASES

**Phase 1: Setup**

The first phase of implementation involves setting up the development environment required for building the system. This includes installing essential tools and technologies such as Python and Node.js, which are used for backend and frontend development respectively. Additionally, the development environment is configured by setting up necessary libraries, dependencies, and virtual environments to ensure smooth execution of the application.

**Phase 2: Frontend Development**

The second phase of implementation focuses on frontend development, where the user interface of the system is designed and developed using React.js. This phase involves creating interactive and dynamic UI components and designing various pages required for the application, such as registration, dashboard, and attendance views. The goal is to ensure a user-friendly and responsive interface that enhances user experience.

**Phase 3: Backend Development**

The third phase involves backend development, where server-side functionalities are implemented using Flask. This includes developing RESTful APIs to handle data processing, user management, and communication between the frontend and backend. Additionally, the database is integrated to store and manage user information and attendance records efficiently.

**Phase 4: AI Integration**

In the fourth phase, Artificial Intelligence functionalities are integrated into the system to enable face recognition capabilities. This includes implementing face detection using computer vision techniques and developing recognition mechanisms using deep learning

models. The system is trained to accurately identify individuals by comparing facial features with stored datasets.

**Phase 5: Testing**

The final phase of implementation focuses on testing the system to ensure its reliability and performance. This involves debugging errors, validating functionalities, and verifying that all components of the system work as expected. Comprehensive testing is carried out to ensure accuracy, efficiency, and smooth operation in real-time scenarios.

## 3.8 TESTING METHODOLOGY

Testing ensures system reliability and accuracy.

**Types of Testing:**

**Unit Testing**

Unit testing is performed to evaluate individual modules of the system independently. Each component, such as user registration, API endpoints, and database operations, is tested separately to ensure that it functions correctly and produces the expected results.

**Integration Testing**

Integration testing focuses on verifying the interaction between different modules of the system. It ensures that the frontend, backend, database, and AI components work together seamlessly and that data is exchanged correctly between them.

**System Testing**

System testing is conducted on the complete application to validate its overall functionality and performance. This type of testing ensures that all components of the system operate together as intended and meet the specified requirements.

**Camera Testing**

Camera testing is carried out to evaluate the real-time face detection and recognition capabilities of the system. It ensures that the camera accurately captures facial data and that the system can reliably recognize individuals and mark attendance under real-world conditions.

## 3.9 DEPLOYMENT

After testing, the system is deployed:

After successful testing, the system is deployed to make it accessible for users. The deployment process begins by running the backend server, which handles data processing and API communication. Next, the frontend application is launched to provide the user interface for interaction. The database is then connected to ensure proper storage and retrieval of user and attendance data. Finally, the complete system is accessed through a web browser, allowing users to interact with the application in a real-time environment.

## 3.10 SECURITY MEASURES

Security measures are implemented to ensure the safety and integrity of the system and its data. Input validation is applied to prevent invalid or malicious data from being processed by the system. Secure API communication is established to protect data exchanged between the frontend and backend. An authentication system is incorporated to ensure that only authorized users can access and manage the application. Additionally, data protection mechanisms are implemented to safeguard sensitive information, ensuring confidentiality and preventing unauthorized access.

## 3.11 Performance Optimization

Performance optimization techniques are implemented to ensure the system operates efficiently and delivers fast results. Efficient API calls are designed to minimize data transfer time and improve communication between system components. Fast face detection algorithms are utilized to enable quick identification of individuals in real-time. Additionally, database

queries are optimized to ensure rapid data retrieval and storage. These improvements collectively contribute to reduced processing time, enhancing the overall performance and responsiveness of the system.

## 3.12 ADVANTAGES OF THE METHODOLOGY

The adopted methodology offers several advantages that contribute to the successful development of the system. It follows a structured and systematic development process, ensuring that each phase is completed efficiently and effectively. The modular approach allows for easy debugging and maintenance, as individual components can be tested and improved independently. Additionally, the system architecture is designed to be scalable, enabling future enhancements and expansion without major changes. The methodology also supports efficient integration of Artificial Intelligence and web technologies, resulting in a robust and high-performing application.

# CHAPTER 4

# PROJECT MANAGEMENT

## 4.1 Internship Timeline

Project management plays a crucial role in ensuring that the internship project is completed within the specified time, budget, and quality standards. It involves planning, scheduling, risk management, and resource allocation.

The AI Face Recognition Attendance Management System project was carried out in a structured manner during the internship at Rail Wheel Factory (RWF), Bengaluru. The work was divided into multiple phases to ensure smooth development and proper execution.

**Timeline Breakdown:**

**Phase 1 (Jan 07 – Apr 30): Requirement Analysis**

- Understanding project requirements
- Studying AI and face recognition basics

**Phase 2 (Feb 01 – Feb 15): System Design**

- Designing architecture
- Planning modules
- Preparing workflow

**Phase 3 (Feb 16 – Mar 16): Development**

- Frontend development using React.js
- Backend development using Flask
- Dataset collection

**Phase 4 (Mar 17 – Mar 30): AI Integration & Testing**

- Implementing face recognition
- Testing system modules
- Debugging errors

**Phase 5 (Mar 30 - Apr 07): Deployment & Finalization**

- Final testing
- System deployment
- Report preparation

**Internship Timeline Visualization**

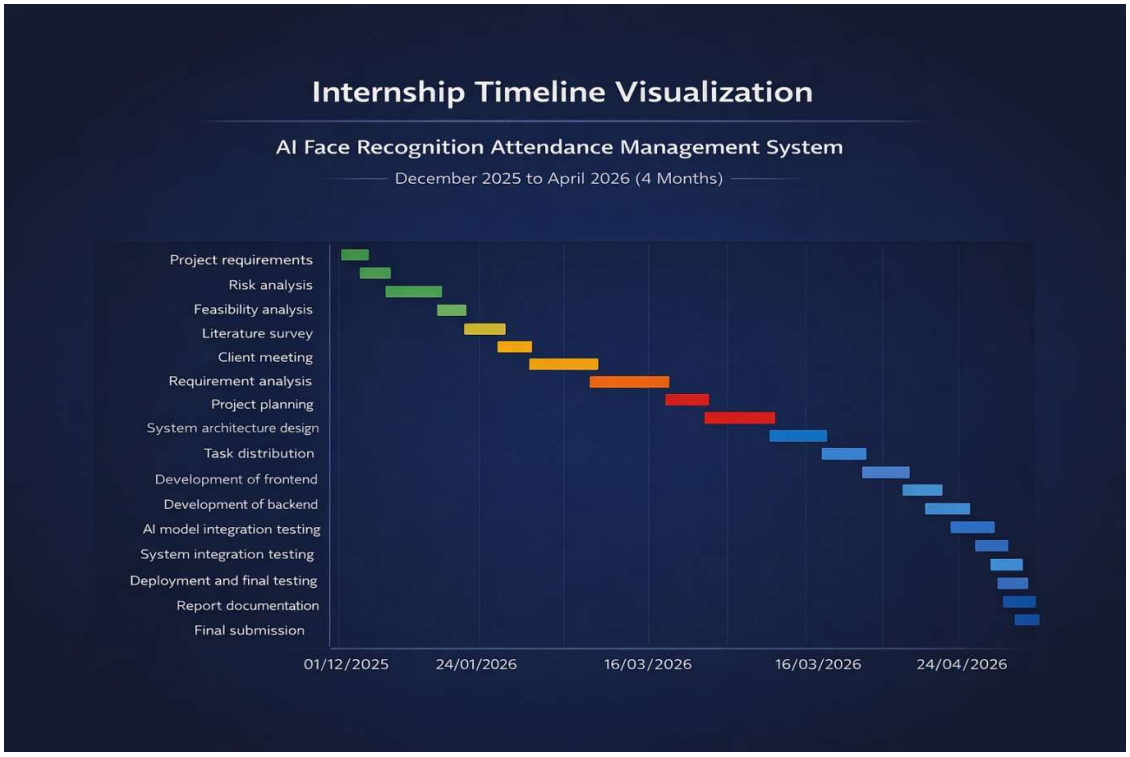
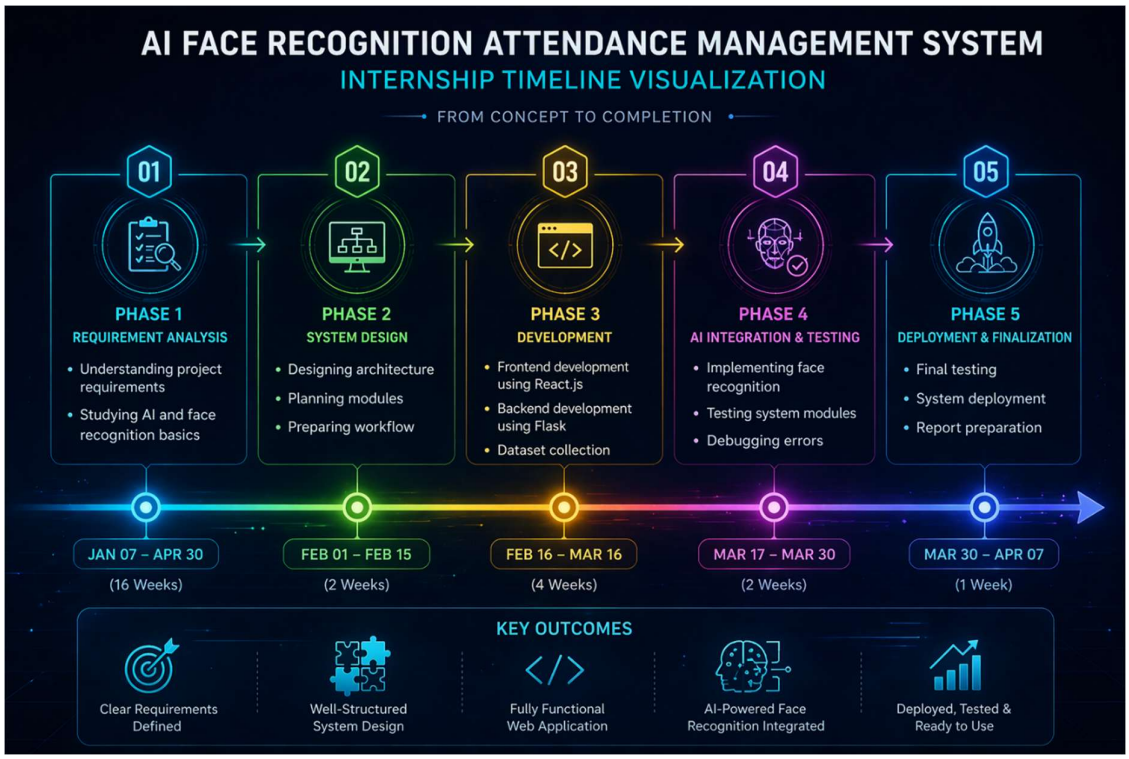



Fig : 4.1




The internship timeline was systematically divided into multiple phases to ensure a structured and efficient development process. The first phase focused on requirement analysis, where a comprehensive understanding of project requirements was developed along with a study of Artificial Intelligence and face recognition fundamentals. This was followed by the system design phase, which involved designing the overall architecture, planning system modules, and preparing a detailed workflow for implementation. The development phase marked the core execution stage, during which the frontend was developed using React.js, the backend was implemented using Flask, and the dataset required for face recognition was collected. Subsequently, the AI integration and testing phase was carried out, where face recognition functionalities were implemented, system modules were thoroughly tested, and errors were

identified and resolved through debugging. Finally, the deployment and finalization phase ensured the system was fully operational through final testing, successful deployment, and preparation of the project report. This phased approach ensured a smooth transition from concept to implementation, resulting in a well-structured and high-performing system.

## 4.2 Risk Analysis

Risk analysis is an important part of project management to identify potential issues and provide solutions.

**Identified Risks and Mitigation:**

Risk analysis helps identify potential issues and provides solutions to overcome them.

**Identified Risks and Solutions:**

**1. Technical Risk**

Difficulty in AI model integration
*Solution:* Use proper documentation and testing

**2. Environment Setup Risk**

Errors during installation of libraries (OpenCV, InsightFace)
*Solution:* Use virtual environment and correct versions

**3. Performance Risk**

Slow face recognition or delay
*Solution:* Optimize code and reduce processing load

**4. Data Risk**

Loss of attendance data
*Solution:* Backup database regularly

**5. Security Risk**

Unauthorized access

*Solution:* Implement authentication and validation

**6. Time Management Risk**

Delay in completing project

*Solution:* Follow timeline strictly and divide tasks

## 4.3 Internship budget

**The project is designed to be** low-cost and student-friendly **by using open-source tools and free services.**

**Table 4.3 Budget Details**

| **Component** | **Description** | **Estimated Cost (INR)** |
|---|---|---|
| Development Tools | VS Code, GitHub (Free tools) | ₹0 |
| Database | SQLite (Local) | ₹0 |
| Hosting | Localhost / Free Hosting | ₹0 – ₹500 |
| AI Libraries | OpenCV, InsightFace (Open Source) | ₹0 |
| Internet Charges | Development & Testing | ₹500 – ₹1000 |
| Miscellaneous | Utilities, Testing | ₹200 – ₹500 |
| **Total Estimated Cost** | | **₹700 – ₹2000 (Approx.)** |

**Explanation**

The project is designed to be low-cost by utilizing free-tier services and open-source technologies. Most of the development tools such as Visual Studio Code and GitHub are freely available. SQLite (Local) provides a free database service, and hosting platforms like Vercel and Render also offer free plans.

AI API usage is kept within free limits, minimizing additional costs. The primary expense is internet usage required for development, testing, and deployment. Overall, the project is highly economical and suitable for student-level implementation.

# CHAPTER 5

# ANALYSIS AND DESIGN

## 5.1 Requirements

The system requirements define what the system should do and how it should perform.

### 5.1.1 Functional Requirements

The functional requirements define the essential operations that the system must perform to ensure effective attendance management. The system should support user registration and management, allowing administrators to add and manage individuals within the system. It must enable face dataset capture using a webcam, followed by face encoding and training to prepare the recognition model. Additionally, the system should perform real-time face detection and recognition to accurately identify users. Once recognized, attendance should be marked automatically and stored securely in the database. The system should also provide features for viewing attendance records and generating reports for analysis. Furthermore, a dashboard interface should be included to facilitate efficient monitoring and management of users and attendance data.

### 5.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must satisfy to ensure optimal performance and usability. The system is expected to deliver high accuracy in face recognition to maintain reliable attendance records. It should provide fast processing speed to support real-time operations without delays. Secure data handling is essential to protect sensitive user information and prevent unauthorized access. Additionally, the system should offer a user-friendly interface to ensure ease of use for both administrators and users. The design must also be scalable, allowing the system to handle an increasing number of users efficiently without compromising performance.

## 5.2 Block Diagram

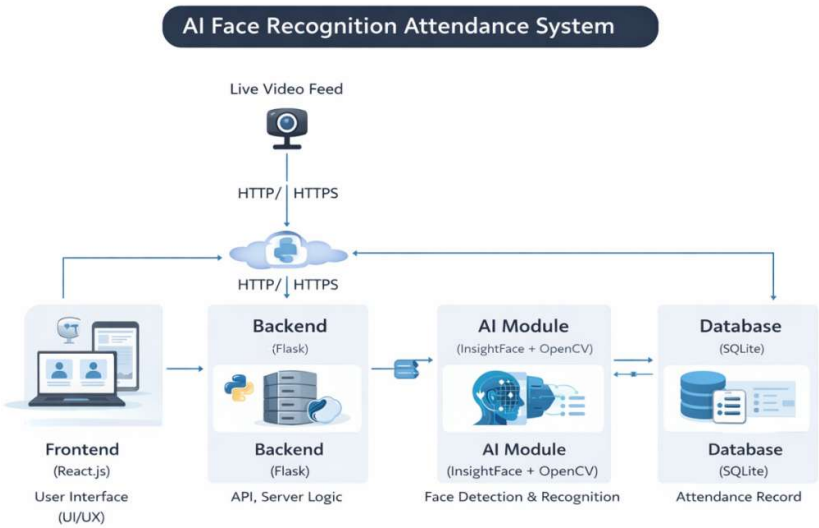



**Fig : 5.2**

**Explanation:**

The system operates through a structured flow of interactions between its components. Initially, the user interacts with the frontend interface to perform actions such as registration or attendance marking. The frontend then sends requests to the backend server through APIs. The backend processes these requests by executing the required logic and utilizing the AI model for face detection and recognition. Once the processing is complete, the relevant data, including attendance records, is stored in the database for future access and management.

## 5.3 System Flow Chart

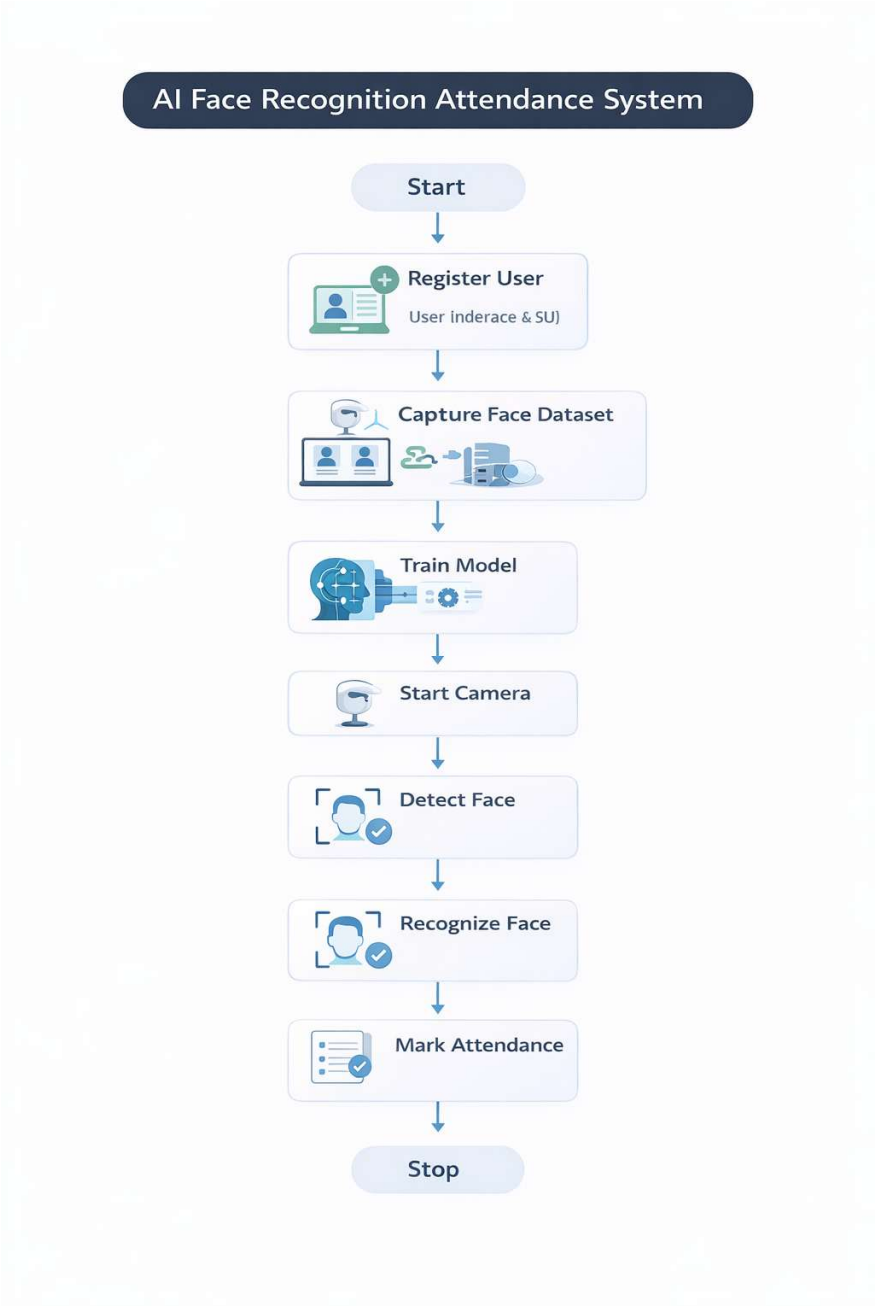



**Fig: 5.3**

**Explanation**

The flowchart of the AI Face Recognition Attendance System illustrates the complete working process from user registration to attendance marking. The process begins with the start phase, where the system is initiated. In the first step, the user is registered through the interface by providing necessary details. Once registered, the system proceeds to capture the face dataset using a webcam, collecting multiple facial images required for training. These images are then used in the model training phase, where the system learns to recognize unique facial features. After training, the system activates the camera to begin real-time operations. It detects faces from the live video stream using computer vision techniques and then performs face recognition by comparing the detected face with the trained dataset. If a match is found, the system identifies the individual accurately. Finally, the attendance is automatically marked and stored in the database. The process then ends at the stop stage.

This workflow ensures a seamless, automated, and efficient attendance management process, minimizing manual effort while maintaining high accuracy and reliability.

## 5.4 Choosing Devices

The system requires the following devices:

The system requires certain hardware components to function effectively. A laptop or desktop computer is needed to run the application and handle processing tasks. Additionally, a webcam or integrated camera is required to capture facial images for detection and recognition. An internet connection is also necessary to support communication between system components, facilitate updates, and ensure smooth operation of the application.

## 5.5 Designing Units

The system is divided into modules:

The system is structured into multiple modules to ensure efficient organization and functionality. The frontend module is responsible for providing the user interface through which users interact with the application. The backend module handles the core logic and

API communication, managing data processing and system operations. The AI module performs face recognition tasks, including face detection and identification using advanced algorithms. Additionally, the database module is used for storing and managing user information and attendance records in a structured manner.

## 5.6 Standards

The system follows standard practices:

The system follows established software development practices to ensure efficiency, scalability, and maintainability. It adopts a REST API architecture to facilitate structured and reliable communication between the frontend and backend components. Data exchange is carried out using the JSON format, which is lightweight and widely supported. Secure coding practices are implemented to protect the system from potential vulnerabilities and ensure data integrity. Additionally, the system follows the Model-View-Controller (MVC) design pattern, which promotes a clear separation of concerns and enhances the overall organization and maintainability of the application.

## MVC architecture

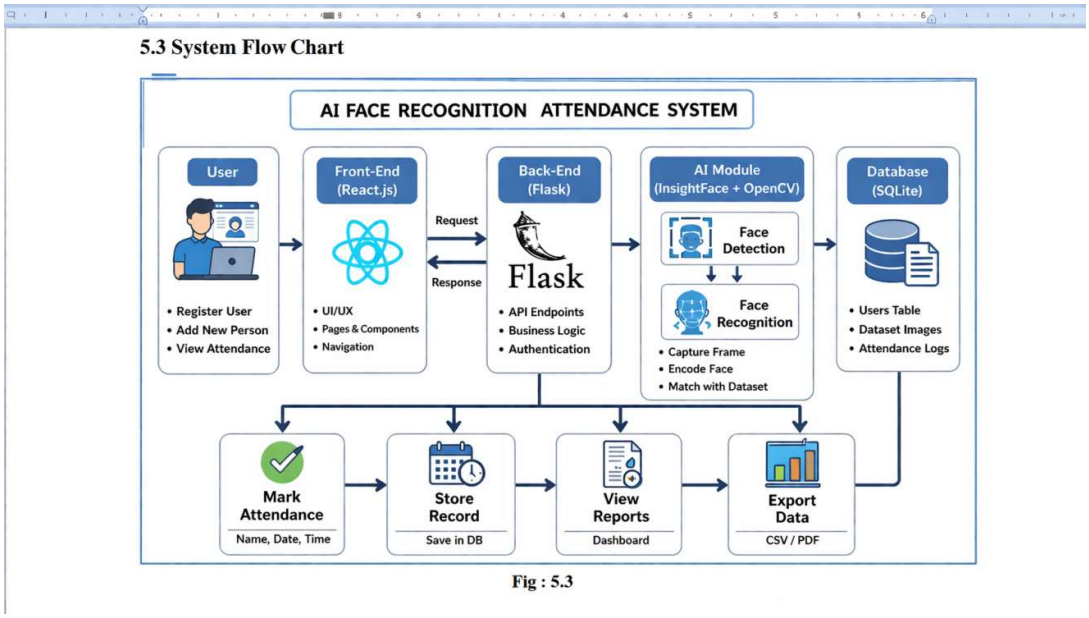



**Fig : 5.6**

**Explanation**

The frontend layer is responsible for handling user input, allowing users to interact with the system through the interface. The backend performs the core processing by executing business logic and managing communication between modules. The database component is dedicated to storage, ensuring that user data and attendance records are securely maintained. Additionally, the AI module provides intelligent processing by performing face detection and recognition, enabling automated and accurate attendance management.

## 5.7 Mapping with Reference Model Layers

(Conceptual mapping)

Application Layer → User Interface

Processing Layer → Backend + AI

Data Layer → Database

## 5.8 Domain Model Specification

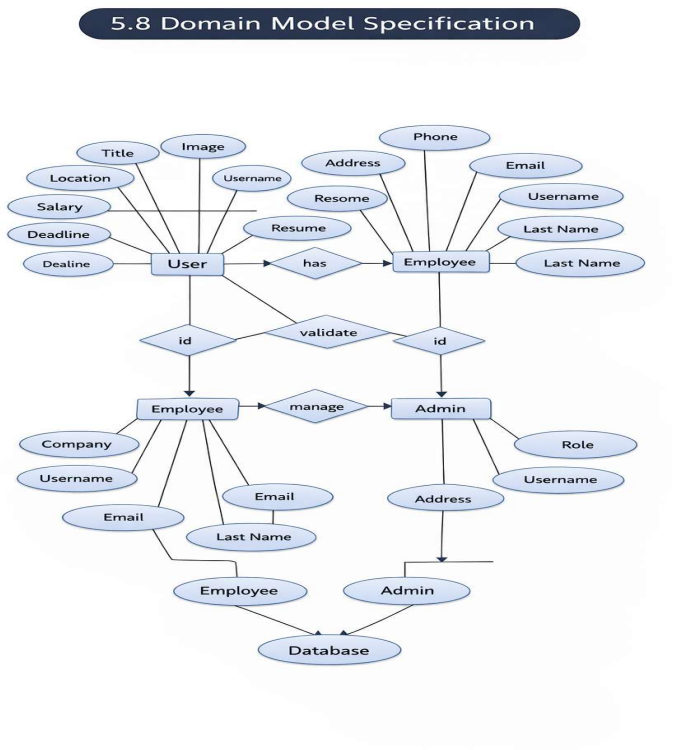



Fig 5.8

**Explanation:**

Entities include User, Resume, Skills, and Experience. A user can create multiple resumes.

The domain model specification illustrates the structure of the system by defining key entities, their attributes, and the relationships between them. At the core of the model is the **User** entity, which contains attributes such as title, image, username, location, salary, and deadline, representing general user-related information. The User entity is associated with the **Employee** entity through a relationship that indicates that a user can have employee-specific details such as address, phone number, email, and resume.

The model also includes an **Admin** entity, which is responsible for managing the system. The Admin is linked to the Employee through a management relationship, indicating that administrators oversee employee records and operations. Both Employee and Admin entities are associated with unique identifiers (ID) and participate in validation processes to ensure data integrity.

Additional attributes such as company details, email, username, and last name are maintained within the Employee entity, while the Admin entity includes attributes like role, username, and address for administrative control. All these entities are ultimately connected to the Database, where the system stores and manages all user, employee, and administrative data.

Overall, this domain model provides a clear representation of how different components of the system interact, ensuring organized data management and efficient system functionality.

## 5.9 Communication Model

The system uses:

The communication model of the system is designed to ensure efficient and secure data exchange between different components. The system utilizes HTTP/HTTPS protocols for communication, with HTTPS providing an added layer of security through encrypted data transmission. It follows a REST API architecture, which enables structured and scalable interaction between the frontend and backend. Data is exchanged in JSON format, which is lightweight, easy to parse, and widely supported across platforms. This communication model ensures reliable, fast, and secure transmission of information within the system.

## 5.10 Deployment Level

The system is a cloud-based web application**.**

## 5.11 Functional View

**Fig : 5.11**

**Explanation:**

Shows how input is processed and transformed into output using backend and AI.

The functional view of the system illustrates how different components interact to deliver a complete AI-based face recognition attendance solution. The system is designed as a full-stack

web application, where the client side (frontend), developed using React and Tailwind CSS, provides the user interface for interaction. It includes public pages such as home, about, and login, along with a user dashboard that allows users to register, capture facial data, start the camera, and view attendance. An admin dashboard is also provided to manage users, view reports, and monitor attendance logs.

The frontend communicates with the backend, developed using Flask (Python), through REST APIs. The backend is responsible for handling authentication, user management, and attendance-related operations. It includes modules such as the Auth and User API for login and user handling, the Attendance API for capturing datasets and managing attendance, AI services for face detection and recognition using InsightFace and OpenCV, and admin controls for generating reports and analytics.

The system integrates AI services to perform real-time face detection and recognition, enabling automated attendance marking. All data is stored and managed in the database layer, primarily using SQLite, which maintains user details, face datasets, and attendance records. In some cases, additional storage such as MongoDB may be used for handling larger datasets or analytics.

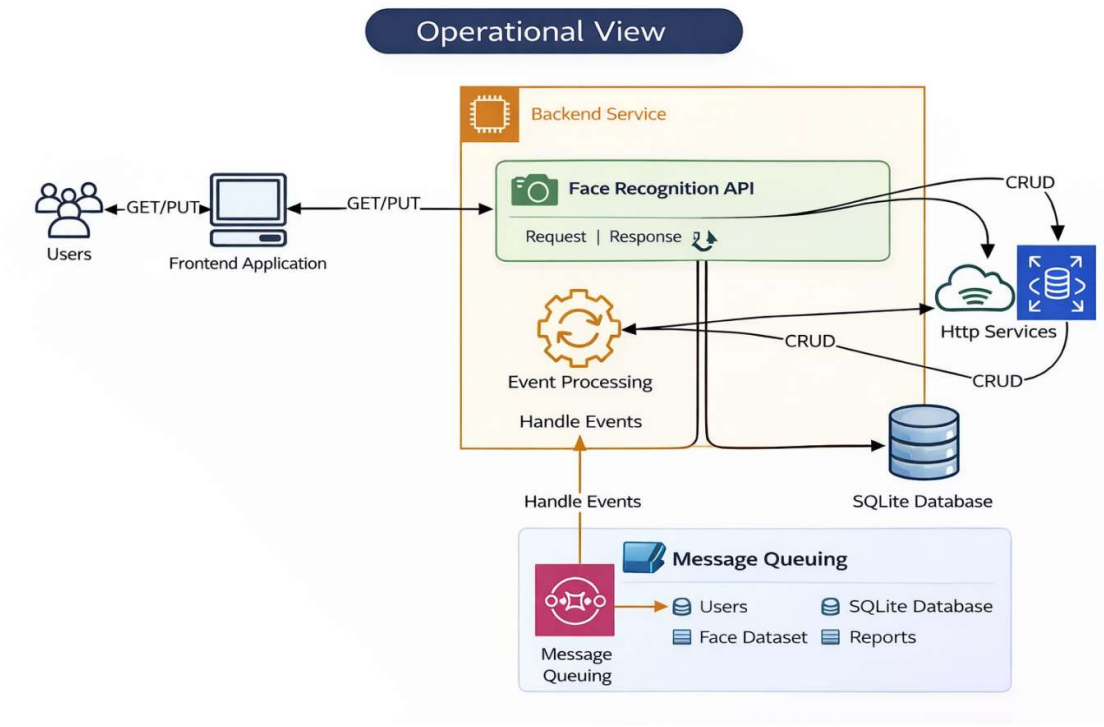
Overall, this functional view demonstrates a well-organized architecture where the frontend handles user interaction, the backend manages processing and logic, the AI module performs intelligent recognition, and the database ensures secure data storage, resulting in an efficient and scalable attendance management system.

## 5.12 Mapping Deployment Level with Functional View

The mapping between the deployment level and the functional view of the system highlights how different components contribute to overall functionality. The frontend layer is responsible for handling user input, allowing users to interact with the system through the interface. The backend performs the core processing by executing business logic and managing communication between modules. The database component is dedicated to storage, ensuring that user data and attendance records are securely maintained. Additionally, the AI module

provides intelligent processing by performing face detection and recognition, enabling automated and accurate attendance management.

## 5.13 Operational View




**5.13 Operational View**

Explanation

The operational view of the system illustrates how different components interact during real-time execution to process user requests efficiently. The process begins with users interacting with the frontend application, which sends requests to the backend service using HTTP methods such as GET and PUT. The backend consists of a Face Recognition API, which handles incoming requests and generates appropriate responses after processing.

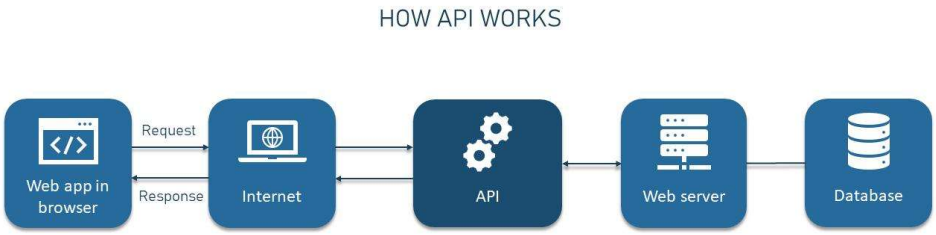
Within the backend, an event processing module manages system operations by handling various events related to face detection, recognition, and attendance marking. The backend communicates with external HTTP services and performs CRUD (Create, Read, Update, Delete) operations to manage data effectively. All essential data, including user details, face

datasets, and attendance records, are stored in the SQLite database, ensuring structured and reliable data management.

Additionally, the system incorporates a message queuing mechanism, which enables asynchronous processing of tasks such as user data handling, face dataset processing, and report generation. This improves system performance by reducing delays and ensuring smooth handling of multiple requests simultaneously.

Overall, the operational view demonstrates a well-coordinated system where the frontend initiates requests, the backend processes them using APIs and event handling, the database manages persistent data storage, and message queuing enhances performance and scalability for real-time operations.

## API Key How Works




**5.13.1 API key works**

**Explanation**

The diagram illustrates the working process of an API (Application Programming Interface) in a web-based system. The process begins with the web application in the browser, where the user performs an action such as logging in or marking attendance. This action generates a request, which is sent over the internet to the API.

The API acts as an intermediary between the frontend and the backend system. It receives the request and forwards it to the web server, where the required processing and business logic are executed. The web server then interacts with the database to retrieve or store the necessary data, such as user details or attendance records.

Once the operation is completed, the server sends the processed data back to the API, which then returns a response to the web application through the internet. Finally, the browser displays the result to the user.

This process ensures efficient communication between the client and server, enabling secure**, structured, and real-time data exchange in modern web applications.**

## 5.14 Other Design

Additional design aspects:

In addition to the core system architecture, several other design aspects are incorporated to enhance usability, security, and performance. The system features a responsive user interface design, ensuring seamless operation across various devices such as desktops, tablets, and mobile screens. Effective error handling mechanisms are implemented to detect and manage unexpected issues, improving system reliability and user experience. Secure authentication is also integrated to ensure that only authorized users can access the system, thereby protecting sensitive data. Furthermore, performance optimization techniques are applied to improve system efficiency, reduce response time, and ensure smooth real-time operation.

# CHAPTER 6

# HARDWARE, SOFTWARE AND SIMULATION

### 6.1 Hardware

The system requires basic hardware components for development and execution.

**Hardware Specifications:**

The system requires basic hardware components to support development and execution of the application. A processor such as Intel i3, i5, or higher is recommended to handle computational tasks efficiently. The system should have a minimum of 4GB RAM, with 8GB preferred for smoother performance, especially during AI model execution. Storage capacity of at least 256GB, either HDD or SSD, is required to store datasets and application files. A webcam, either built-in or external, is essential for capturing real-time video input for face detection and recognition. The system must operate on a 64-bit operating system to ensure compatibility with modern software tools and libraries. Additionally, an internet connection is required for installing dependencies, updates, and supporting certain functionalities. The webcam captures live video data, while the processor and RAM enable efficient processing of AI models, and storage is used to maintain datasets and attendance records.

**Description:**

The **webcam** is used to capture real-time video for face detection.The **processor and RAM** ensure smooth execution of AI models .The **storage** is used to store dataset

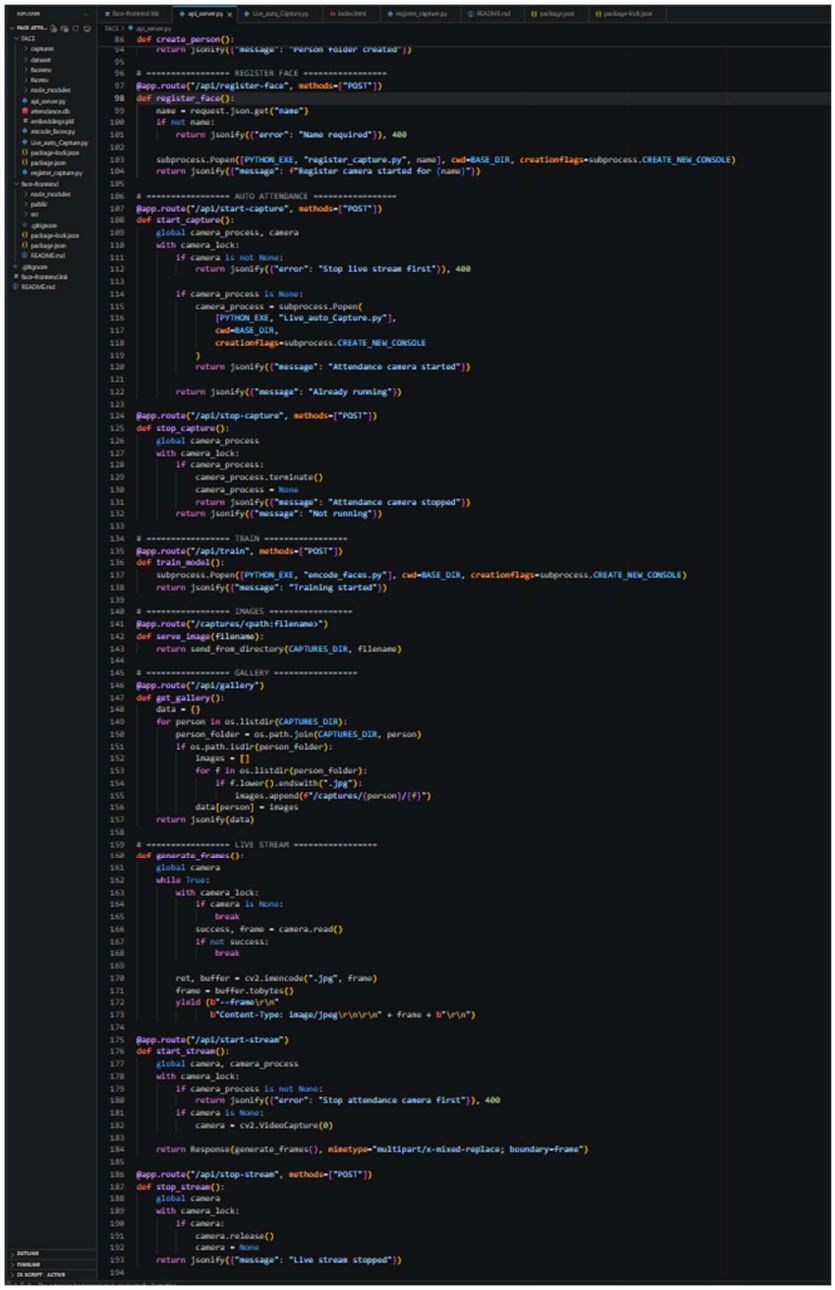
## 6.2 Software Development Tools

The project uses modern software tools for development:

The project utilizes a range of modern software development tools to ensure efficient design, development, and testing of the system. Visual Studio Code is used as the primary code editor,

providing a flexible and feature-rich environment for development. React.js is employed for frontend development to create an interactive and responsive user interface. Node.js and Express.js are used for backend development to handle server-side logic and API communication. MongoDB is utilized for database management, offering scalability and flexibility in handling data. Postman is used for testing APIs and verifying communication between system components. Additionally, GitHub is used for version control, enabling code management, collaboration, and tracking of project progress.

## 6.3 Software Code Snippets



```python
def create_person():
    return jsonify({"message": "Person folder created"})

# ================ REGISTER FACE ================
@app.route("/api/register-face", methods=["POST"])
def register_face():
    name = request.json.get("name")
    if not name:
        return jsonify({"error": "Name required"}), 400

    subprocess.Popen([PYTHON_EXE, "register_capture.py", name], cwd=BASE_DIR, creationflags=subprocess.CREATE_NEW_CONSOLE)
    return jsonify({"message": f"Register camera started for {name}"})

# ================ AUTO ATTENDANCE ================
@app.route("/api/start-capture", methods=["POST"])
def start_capture():
    global camera_process, camera
    with camera_lock:
        if camera is not None:
            return jsonify({"error": "Stop live stream first"}), 400

        if camera_process is None:
            camera_process = subprocess.Popen(
                [PYTHON_EXE, "Live_auto_Capture.py"],
                cwd=BASE_DIR,
                creationflags=subprocess.CREATE_NEW_CONSOLE
            )
            return jsonify({"message": "Attendance camera started"})

        return jsonify({"message": "Already running"})

@app.route("/api/stop-capture", methods=["POST"])
def stop_capture():
    global camera_process
    with camera_lock:
        if camera_process:
            camera_process.terminate()
            camera_process = None
            return jsonify({"message": "Attendance camera stopped"})
        return jsonify({"message": "Not running"})

# ================ TRAIN ================
@app.route("/api/train", methods=["POST"])
def train_model():
    subprocess.Popen([PYTHON_EXE, "encode_faces.py"], cwd=BASE_DIR, creationflags=subprocess.CREATE_NEW_CONSOLE)
    return jsonify({"message": "Training started"})

# ================ IMAGES ================
@app.route("/captures/<path:filename>")
def serve_image(filename):
    return send_from_directory(CAPTURES_DIR, filename)

# ================ GALLERY ================
@app.route("/api/gallery")
def get_gallery():
    data = {}
    for person in os.listdir(CAPTURES_DIR):
        person_folder = os.path.join(CAPTURES_DIR, person)
        if os.path.isdir(person_folder):
            images = []
            for f in os.listdir(person_folder):
                if f.lower().endswith(".jpg"):
                    images.append(f"/captures/{person}/{f}")
            data[person] = images
    return jsonify(data)

# ================ LIVE STREAM ================
def generate_frames():
    global camera
    while True:
        with camera_lock:
            if camera is None:
                break
            success, frame = camera.read()
            if not success:
                break

        ret, buffer = cv2.imencode(".jpg", frame)
        frame = buffer.tobytes()
        yield (b"--frame\r\n"
               b"Content-Type: image/jpeg\r\n\r\n" + frame + b"\r\n")

@app.route("/api/start-stream")
def start_stream():
    global camera, camera_process
    with camera_lock:
        if camera_process is not None:
            return jsonify({"error": "Stop attendance camera first"}), 400
        if camera is None:
            camera = cv2.VideoCapture(0)

    return Response(generate_frames(), mimetype="multipart/x-mixed-replace; boundary=frame")

@app.route("/api/stop-stream", methods=["POST"])
def stop_stream():
    global camera
    with camera_lock:
        if camera:
            camera.release()
            camera = None
    return jsonify({"message": "Live stream stopped"})
```

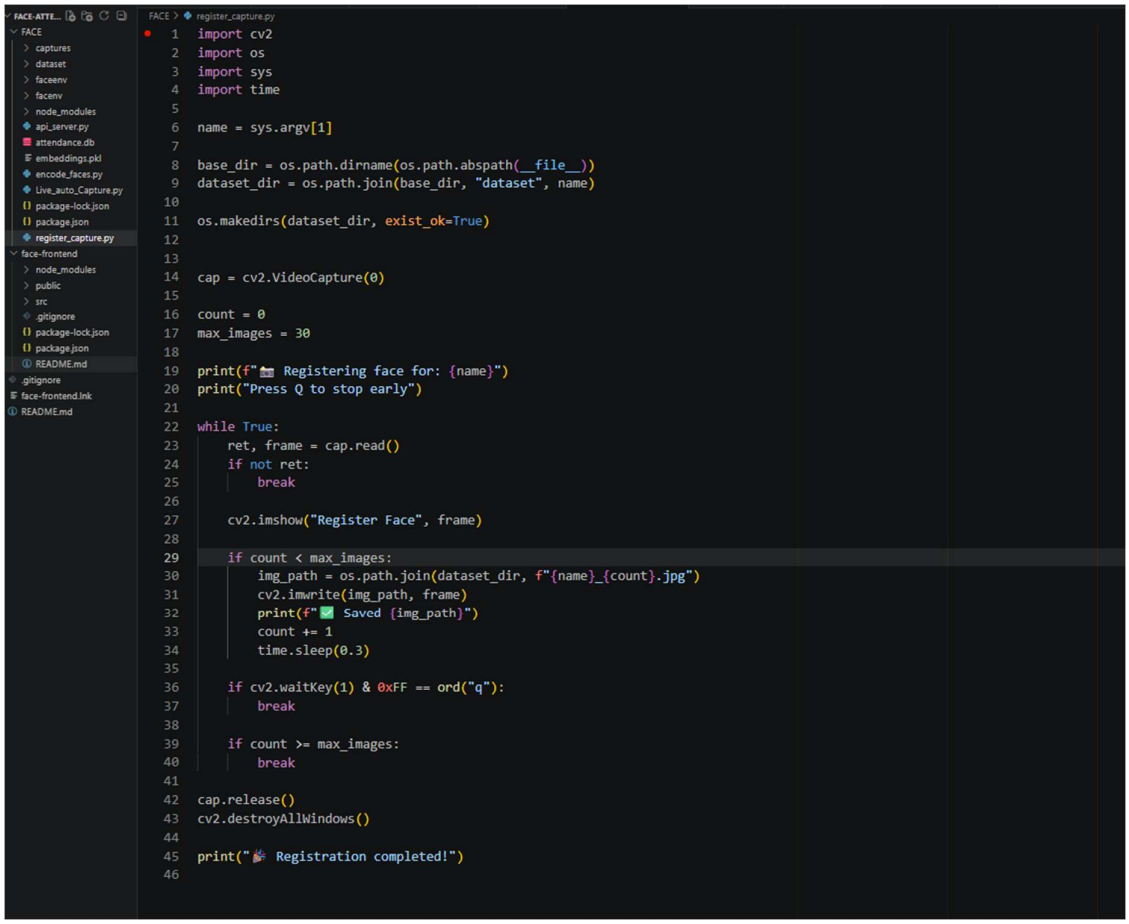
1

```python
import cv2
import os
import pickle
import insightface
import numpy as np

# -------------------- Initialize InsightFace --------------------
app = insightface.app.FaceAnalysis(name="buffalo_l")
app.prepare(ctx_id=0)

# -------------------- Paths --------------------
dataset_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")
embeddings = {}

# -------------------- Check dataset --------------------
if not os.path.exists(dataset_path):
    print("❌ Dataset folder not found!")
    exit()

# -------------------- Loop through each person folder --------------------
for person_name in os.listdir(dataset_path):
    person_folder = os.path.join(dataset_path, person_name)

    if not os.path.isdir(person_folder):
        continue

    print(f"\n📂 Processing person: {person_name}")

    person_embeddings = []

    for file in os.listdir(person_folder):
        if not file.lower().endswith((".jpg", ".png", ".jpeg")):
            continue

        img_path = os.path.join(person_folder, file)

        try:
            img = cv2.imread(img_path)
            if img is None:
                print(f"❌ Cannot read image: {img_path}")
                continue

            faces = app.get(img)
            if len(faces) == 0:
                print(f"⚠ No face found in: {img_path}")
                continue

            # Take first face only
            face = faces[0]

            # Normalize embedding
            emb = face.embedding / np.linalg.norm(face.embedding)

            person_embeddings.append(emb)
            print(f"  ✅ Encoded {file}")

        except Exception as e:
            print(f"❌ Error in {img_path}: {e}")

    # -------------------- Average all embeddings for this person --------------------
    if len(person_embeddings) > 0:
        avg_emb = np.mean(person_embeddings, axis=0)
        avg_emb = avg_emb / np.linalg.norm(avg_emb)

        embeddings[person_name] = avg_emb
        print(f"✅ Saved embedding for {person_name}")
    else:
        print(f"❌ No valid faces for {person_name}")

# -------------------- Save embeddings --------------------
with open("embeddings.pkl", "wb") as f:
    pickle.dump(embeddings, f)

print("\n🎉 Face encoding completed!")
print(f"📊 Total persons encoded: {len(embeddings)}")
```

2

```python
# -------------------- Video Capture --------------------
cap = cv2.VideoCapture(0)
THRESHOLD = 0.65
last_seen = {}     # for DB duplicate control
last_spoken = {}   # for voice / alarm control

print("Press Q to quit")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    faces = app.get(frame)

    screen_message = "No Face Detected"
    screen_color = (0, 255, 255)

    for face in faces:
        emb = face.embedding
        name = "Unknown"
        min_dist = 1.0

        # -------------------- Check for known faces --------------------
        for person, known_emb in known_embeddings.items():
            dist = cosine(emb, known_emb)
            if dist < min_dist and dist < THRESHOLD:
                min_dist = dist
                name = person

        # -------------------- Draw box --------------------
        x1, y1, x2, y2 = map(int, face.bbox)
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(frame, name, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)

        # -------------------- Welcome / Unknown Message --------------------
        now = datetime.now()

        if name != "Unknown":
            screen_message = f"Welcome {name}"
            screen_color = (0, 255, 0)

            # 🔊 Speak only once every 20 seconds per person
            if name not in last_spoken or (now - last_spoken[name]).seconds > 20:
                last_spoken[name] = now
                engine.say(f"Welcome {name}")
                engine.runAndWait()

        else:
            screen_message = "Unknown Person"
            screen_color = (0, 0, 255)

            # 🚨 Beep alarm every 10 seconds for unknown
            if "Unknown" not in last_spoken or (now - last_spoken["Unknown"]).seconds > 10:
                last_spoken["Unknown"] = now
                winsound.Beep(1000, 700)  # frequency, duration

        # -------------------- Date & Time --------------------
        date_str = now.strftime("%Y-%m-%d")
        time_str = now.strftime("%H:%M:%S")
        ts_filename = now.strftime("%Y-%m-%d_%H-%M-%S")

        # -------------------- DB & Image save (10 sec rule) --------------------
        if name not in last_seen or (now - last_seen[name]).seconds > 10:
            last_seen[name] = now

            # Create person folder
            person_folder = os.path.join(captures_root, name)
            os.makedirs(person_folder, exist_ok=True)

            # Save image
            file_name = f"{ts_filename}.jpg"
            file_path = os.path.join(person_folder, file_name)
            cv2.imwrite(file_path, frame)
```

3




4

## Explanation

The provided code represents the backend implementation of the AI Face Recognition Attendance System using the Flask framework. It defines multiple REST API endpoints that handle core functionalities such as user registration, face dataset capture, camera control, model training, image retrieval, and live video streaming.

The user registration module is implemented through an API endpoint that accepts user input (such as name) in JSON format and validates it before initiating the face capture process. The system uses subprocess execution to trigger external scripts responsible for capturing facial data, ensuring modular and organized processing.

The camera control functionality includes APIs to start and stop attendance capture. When the camera is activated, the system initializes a video stream using OpenCV and ensures that

no conflicting processes are running simultaneously. Proper error handling is implemented to prevent multiple camera instances or invalid operations.

The training module is responsible for initiating the face recognition model training process by executing a separate script. This allows the system to learn facial features from the collected dataset and prepare for accurate recognition.

The code also includes image handling APIs, which allow retrieval of stored images and gallery data from the dataset directory. This enables visualization and verification of captured face data.

A live streaming module is implemented using a generator function that continuously captures frames from the camera, encodes them into JPEG format, and streams them as a response. This supports real-time monitoring of face detection and recognition.

Overall, this backend implementation follows a modular and API-driven architecture, ensuring efficient communication between frontend and backend, proper handling of AI processes, and smooth real-time attendance tracking.

The software code is written using JavaScript for both frontend and backend.

The software code for the system is primarily developed using JavaScript for both frontend and backend components, ensuring consistency and seamless integration across the application. The frontend code is responsible for handling the user interface and managing user interactions, providing a responsive and intuitive experience. The backend code manages APIs and server-side logic, enabling efficient processing of requests and communication between system components. Additionally, the AI integration code is used to generate intelligent outputs, such as resume content, based on processed data. The database-related code handles the storage, retrieval, and management of data, ensuring that information is securely maintained and efficiently accessed when required.

## 6.4 Simulation

**Working Simulation:**

1. User opens the application
2. Registers a new person
3. Captures face dataset using webcam

4. System trains the model
5. Live camera is started
6. Face is detected and recognized
7. Attendance is marked automatically

**ADMIN LOGIN**

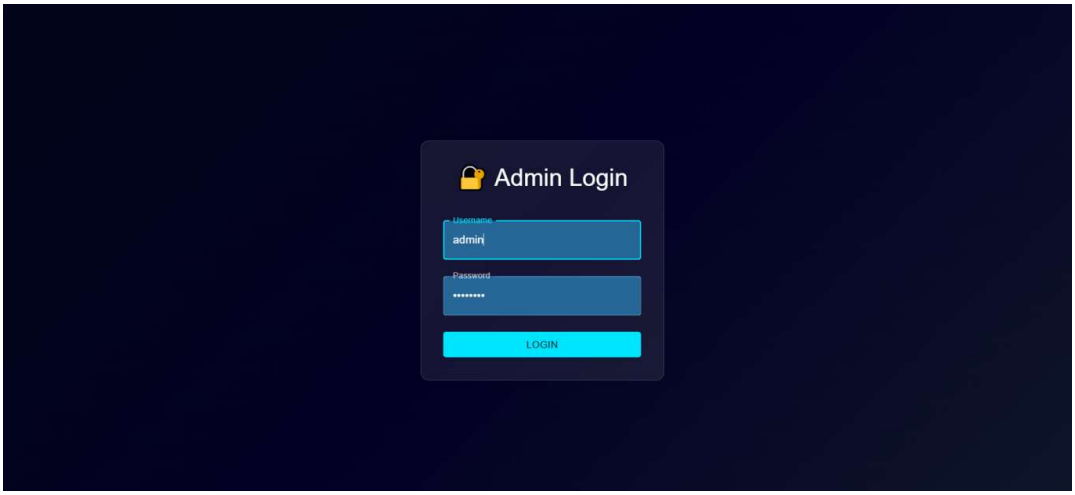



**Fig 6.4.1**

**DASHBOARD**

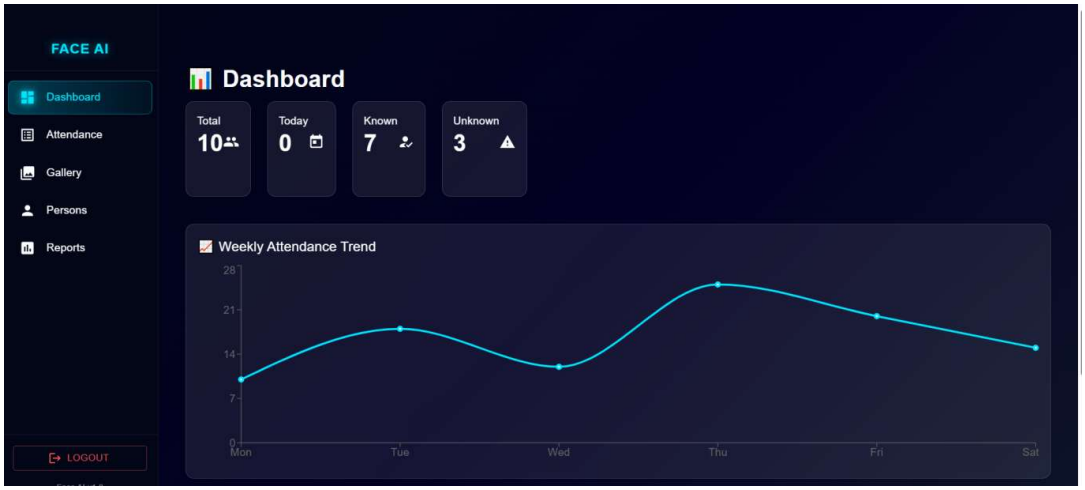



**Fig 6.4.2**

**ATTENDANCE PAGE**

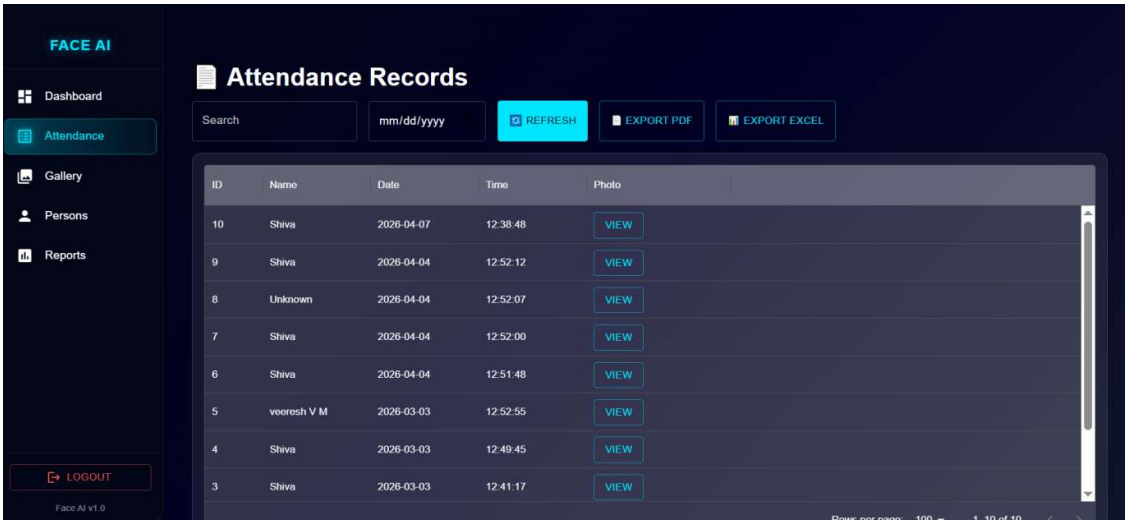



**Fig 6.4.3**

## GALLERY PAGE

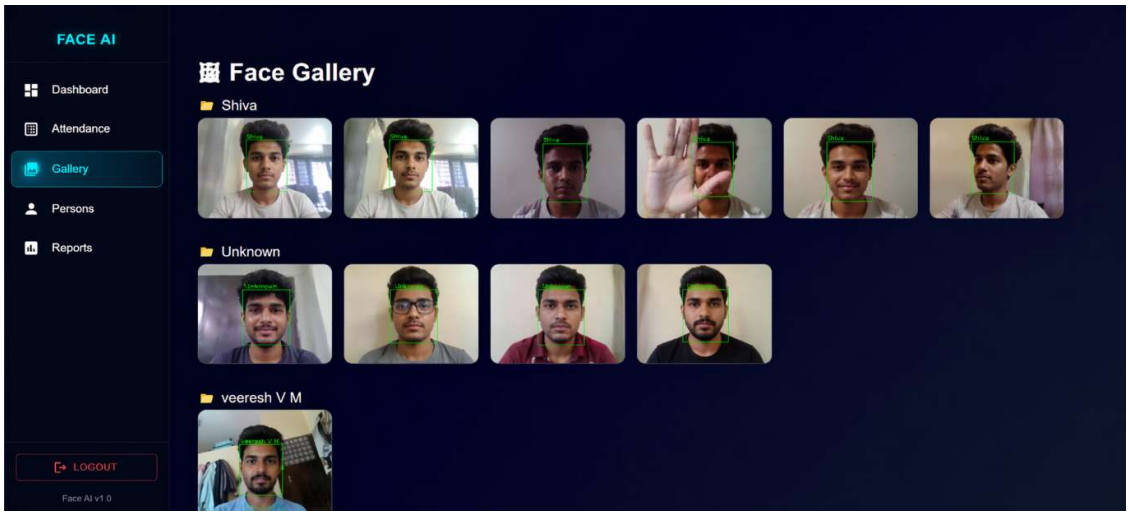



**Fig 6.4.4**

## PERSON PAGE

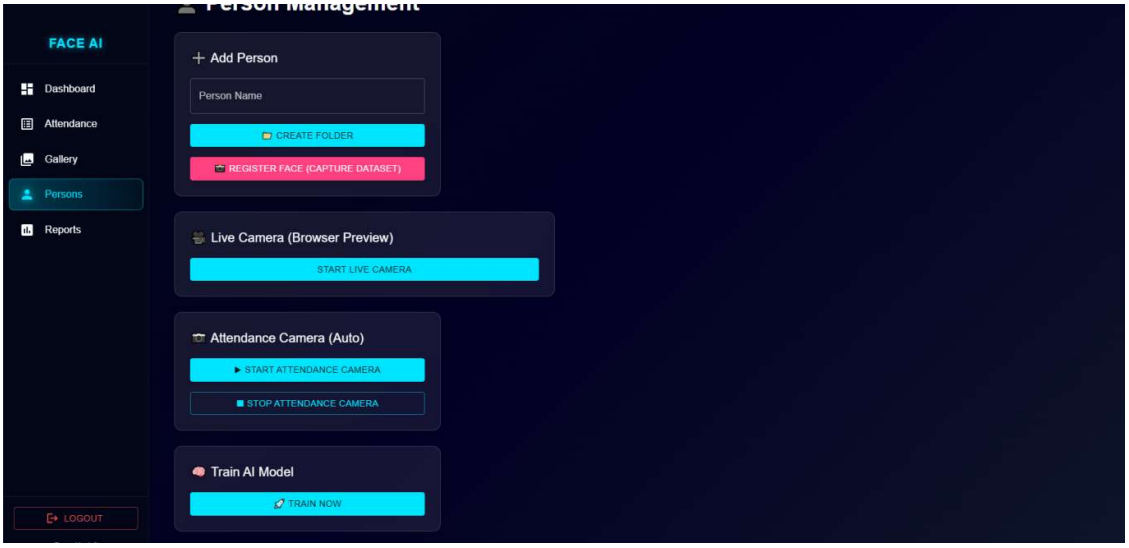



**Fig 6.4.5**

## REPORT PAGE

**Fig 6.4.6**

# CHAPTER 7
# EVALUATION AND RESULTS

## 7.1 Test Points

Test points define the key areas where the system is tested to ensure proper functionality.

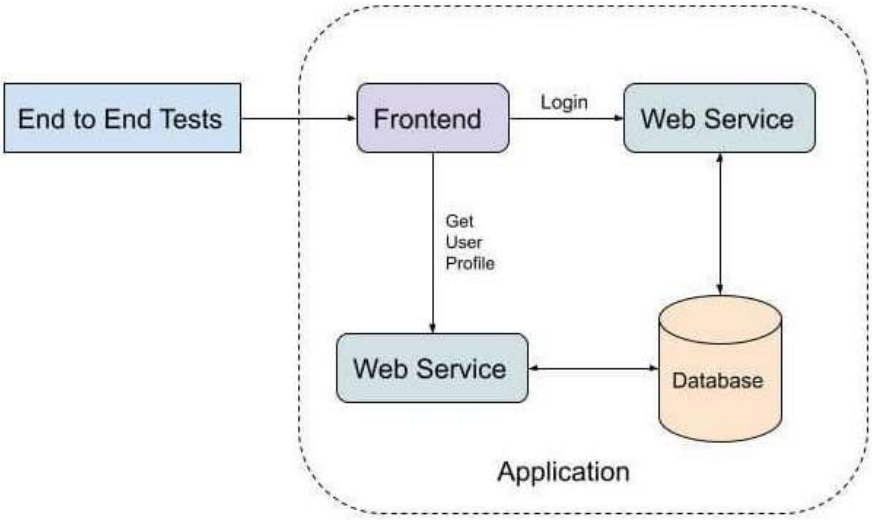



**Fig : 7.1 Test Points**

Explanation

The diagram illustrates the interaction between different components of the application during end-to-end testing. The process begins with End-to-End Tests, which simulate real user actions to validate the complete system workflow. These tests interact with the frontend, which serves as the user interface of the application.

The frontend communicates with the web service (backend) by sending requests such as login and fetching user profile data. The web service processes these requests and interacts with the database to retrieve or store relevant information. For example, during login, the backend verifies user credentials by accessing the database, and for profile requests, it fetches the corresponding user data.

The responses generated by the web service are sent back to the frontend, which then displays the results to the user. This entire flow ensures that all components—frontend, backend, and database—are functioning correctly together.

Overall, the diagram demonstrates how end-to-end testing validates the complete application by simulating real-world scenarios, ensuring that data flows correctly across all layers and that the system performs reliably under actual usage conditions.

**Key Test Points:**

**Functional Test Points:**

**User Registration Process**

The user registration process is tested to ensure that new users can be added successfully into the system. This includes validating input fields, storing user details correctly, and ensuring that the system responds appropriately to both valid and invalid inputs.

**Face Dataset Capture**

This test verifies that the system can accurately capture facial images using the webcam. It ensures that multiple images are collected properly and stored in the dataset for further processing.

**Model Training**

The model training functionality is tested to confirm that the system can process the captured dataset and generate a trained face recognition model without errors. It also checks whether the training process completes within a reasonable time.

**Face Detection Accuracy**

Face detection accuracy is evaluated to ensure that the system can correctly identify the presence of a face in real-time video streams under various conditions.

**Face Recognition Accuracy**

This test ensures that the system can correctly match detected faces with stored data. It evaluates how accurately the system identifies registered users and minimizes false matches.

**Attendance Marking**

The attendance marking feature is tested to verify that attendance is recorded automatically when a recognized face is detected. It also ensures that duplicate entries are avoided and records are stored correctly.

**Non-Functional Test Points**

**System Performance**

System performance is tested to evaluate how efficiently the application operates under normal and heavy workloads. It ensures that the system remains stable and responsive during continuous usage.

**Response Time**

Response time testing measures how quickly the system processes requests and returns results. It ensures minimal delay in operations such as face recognition and attendance marking.

**Accuracy Under Different Lighting Conditions**

This test evaluates the system's ability to maintain recognition accuracy under varying lighting conditions, such as low light, bright light, and shadows.

**Scalability for Multiple Users**

Scalability testing ensures that the system can handle multiple users simultaneously without performance degradation. It verifies the system's capability to expand as the number of users increases.

**Reliability**

Reliability testing ensures that the system consistently performs its intended functions without failure over time. It checks for system stability, error handling, and the ability to recover from unexpected issues.

## 7.2 Test Plan

The test plan defines how testing is performed to validate the system.

**Testing Strategy:**

| Test Case | Description | Expected Result |
|---|---|---|
| | **Register new user**<br>→ New user is successfully registered into system. | New user is successfully registered into the system. |
| TC1 | **Capture face dataset**<br>→ High-quality face images are captured and stored for training. | High-quality face images are captured and stored for training. |
| TC2 | **Train model**<br>→ Face recognition model is trained without any errors. | Face recognition model is trained without any errors. |
| TC3 | **Detect face**<br>→ Face is accurately detected from live camera feed. | Face is accurately detected from live camera feed. |
| TC4 | **Recognize face**<br>→ Registered person is accurately identified from the dataset. | Registered person is accurately identified from the dataset. |
| TC5 | **Mark attendance**<br>→ Attendance is correctly marked with date and time. | Attendance is correctly marked with date and time. |

**Fig : 7.2 Test Plan**

Explanation

The testing strategy diagram presents a structured approach used to validate the core functionalities of the AI Face Recognition Attendance System. It outlines a series of test cases, each associated with a specific module of the system, along with their descriptions and expected outcomes.

The first test case focuses on user registration, ensuring that new users can be successfully added to the system without errors. The next step involves capturing the face dataset, where high-quality facial images are collected and stored for training purposes. Following this, the model training test case verifies that the face recognition model is trained accurately using the captured dataset.

Subsequent test cases evaluate the system's real-time capabilities. The face detection test ensures that faces are correctly identified from the live camera feed, while the face recognition test confirms that registered users are accurately matched with stored data. Finally, the attendance marking test verifies that attendance is recorded correctly with proper date and time details.

Each test case is validated with expected results, indicating successful execution of the system functionalities. This structured testing approach ensures that all critical components of the system are functioning correctly, resulting in a reliable and efficient attendance management **solution.**

**Testing Methods Used:**

Various testing methods were employed to ensure the correctness and reliability of the system. Unit testing was conducted to verify individual components such as login functionality, form inputs, and API calls, ensuring that each module operates correctly in isolation. Integration testing was performed to evaluate the interaction between different system components, including the frontend, backend, database, and AI modules, ensuring seamless data flow and

communication. System testing was carried out on the complete application to validate its overall functionality and performance as a fully integrated system.

The testing environment consisted of widely used tools and platforms to simulate real-world usage conditions. The application was tested using the Chrome browser to ensure compatibility and proper user interface behavior. A localhost server was used to run and test the application during development. Additionally, Postman was utilized for API testing, allowing verification of request-response cycles and ensuring proper backend functionality.

**Testing Environment:**

The testing environment for the system was set up using reliable and widely used tools to ensure accurate validation of functionalities. The application was tested using the Chrome browser to verify user interface behavior and compatibility. A localhost server was used to run and monitor the application during development and testing phases. Additionally, Postman was utilized for API testing, enabling efficient verification of request-response interactions and ensuring proper communication between the frontend and backend components.

## 7.3 Test Result

The system was tested under different conditions, and the results are as follows:

**Results Summary:**

The results obtained from system testing demonstrate the effectiveness and reliability of the proposed solution. The face detection module performs accurately under normal lighting conditions, successfully identifying faces in real-time video streams. The face recognition component achieves high accuracy when working with properly trained datasets, ensuring correct identification of registered users. Additionally, the system exhibits fast response times, enabling smooth real-time operation without noticeable delays. The attendance marking functionality operates correctly, recording entries with accurate date and time details and without any errors. Overall, the results indicate that the system meets its intended objectives and performs efficiently in practical scenarios.

**Performance:**

The performance evaluation of the system indicates strong efficiency and reliability in real-time operations. The face recognition module achieves an accuracy of approximately 90–95%, demonstrating effective identification of registered users under standard conditions. The system exhibits fast response times, enabling real-time processing of face detection and attendance marking without noticeable delays. Additionally, the overall system stability is high, ensuring consistent performance during continuous usage without crashes or failures. These results confirm that the system meets the required performance standards for practical deployment.

**Sample Output:**

The sample output of the system demonstrates its real-time face recognition capabilities. When a face is detected through the camera, the system highlights it with a bounding box, indicating successful detection. Once the face is recognized, the corresponding user's name is displayed on the screen, confirming accurate identification. Simultaneously, the system automatically marks attendance by recording the user's details along with the current date and time. This output reflects the seamless integration of detection, recognition, and attendance logging functionalities within the system.

## 7.4 Insights

The following insights were gained from the project:

Several important insights were gained during the development and implementation of the project. The use of Artificial Intelligence significantly enhances automation in attendance systems, reducing the need for manual intervention. Face recognition technology proves to be effective in minimizing proxy attendance by ensuring accurate identification of individuals. It was also observed that proper dataset collection and training play a crucial role in improving the overall accuracy of the system. Additionally, environmental factors such as lighting conditions have a noticeable impact on recognition performance, highlighting the need for controlled conditions or adaptive algorithms. Furthermore, real-time systems require careful .

# CHAPTER 8

# SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

## 8.1 Social Aspects

The system has a positive impact on society by improving efficiency and reducing manual work.

The proposed system has a significant positive impact on society by enhancing efficiency and reducing reliance on manual processes. By automating attendance marking, it minimizes the time required for traditional methods and streamlines daily operations. The use of face recognition technology effectively eliminates proxy attendance, ensuring authenticity and accountability. This contributes to improved discipline in educational institutions and workplaces, as attendance is monitored accurately and transparently. Furthermore, the system enhances overall productivity by reducing administrative workload and enabling real-time monitoring of attendance data. These benefits collectively demonstrate the system's potential to improve organizational efficiency and promote a more structured and reliable environment.

**Impact:**

The system helps educational institutions and organizations adopt digital transformation and automation, making processes more efficient and reliable.

## 8.2 Legal Aspects

The system involves handling personal data (face images), so legal compliance is important.

**Key Considerations:**

The system involves the collection and processing of personal data, particularly facial images,

making legal compliance a critical consideration. It is essential to obtain explicit user consent before capturing and storing face data to ensure transparency and ethical usage. The collected data must be stored securely using appropriate security measures to prevent misuse or breaches. Additionally, the system should adhere to relevant data protection laws and regulations to safeguard user privacy and maintain compliance standards. Preventing unauthorized access is also a key requirement, which can be achieved through authentication mechanisms and secure system design. These legal considerations ensure that the system operates responsibly while protecting user rights and data integrity.

**Compliance:**

The system should follow basic privacy regulations and ensure that user data is not misused.

## 8.3 Ethical Aspects

Ethical concerns arise when dealing with biometric data and AI systems.

The use of biometric data and Artificial Intelligence in the system raises important ethical considerations that must be carefully addressed. It is essential to respect user privacy by ensuring that personal data, particularly facial information, is collected and used responsibly. Measures should be taken to prevent the misuse of facial data, limiting its use strictly to the intended purpose of attendance management. Transparency in system usage is also crucial, where users are informed about how their data is collected, processed, and stored. Additionally, efforts must be made to prevent bias in AI recognition by using diverse and well-balanced datasets, ensuring fair and accurate identification for all users. Addressing these ethical aspects helps build trust and ensures responsible deployment of the system.

**Responsibility:**

Developers must ensure that the system is used only for legitimate purposes such as attendance and not for surveillance misuse.

## 8.4 Sustainability Aspects

The system is designed to be sustainable and environmentally friendly.

The system is designed with sustainability in mind, promoting environmentally friendly practices through the use of digital technologies. By eliminating the need for paper-based attendance registers, it significantly reduces paper consumption and supports a paperless environment. The use of digital records instead of physical storage further minimizes resource usage and enhances data accessibility. Additionally, the system operates with low hardware requirements, making it energy-efficient and cost-effective. These aspects collectively contribute to reducing the overall environmental impact while encouraging eco-friendly practices in organizations and institutions.

**Impact:**

Promotes eco-friendly practices and reduces environmental impact.

## 8.5 Safety Aspects

Safety ensures that the system is secure and reliable.

Safety is a critical aspect of the system, ensuring that data and system operations remain secure and reliable. The system implements secure database storage to protect sensitive information from potential threats. A robust user authentication mechanism ensures that only authorized users can access the system. Measures are taken to prevent unauthorized access and safeguard biometric data, maintaining confidentiality and integrity. Regular data backups are also performed to prevent data loss and ensure system reliability. Overall, the system is designed to maintain high safety standards by protecting sensitive data and ensuring controlled access for authorized users only.

# Chapter 9

## CONCLUSION

The AI Face Recognition Attendance Management System developed during this internship successfully demonstrates the practical application of Artificial Intelligence and Computer Vision in solving real-world problems. The project provides an efficient and automated alternative to traditional attendance systems, which are often time-consuming and prone to errors. By integrating technologies such as React.js for frontend development, Flask for backend processing, and InsightFace with OpenCV for face recognition, along with a SQLite database, the system ensures smooth functionality and real-time performance. The ability to accurately detect and recognize faces and automatically mark attendance highlights the effectiveness and reliability of the proposed solution.

The internship experience played a crucial role in enhancing both technical and practical knowledge. It provided valuable exposure to full-stack development, API integration, AI model implementation, and real-time system design. Through this project, important skills such as problem-solving, debugging, and system optimization were developed. Additionally, the project emphasized key aspects such as user-friendly design, scalability, and secure data handling. Considerations related to data privacy, ethical use of AI, and system security were also addressed, ensuring responsible and professional implementation of the system.

In the future, the system can be further enhanced by improving recognition accuracy under varying lighting conditions and integrating advanced deep learning models. It can be extended to support cloud-based storage, mobile applications, and large-scale deployments across multiple locations. Additional features such as mask detection, emotion recognition, and integration with IoT devices can further expand its capabilities. The system has strong potential for real-world applications in educational institutions, corporate offices, and industrial environments, contributing to smarter, more efficient, and automated attendance management in the digital era.

# REFERENCES

[1] OpenCV, *"Open Source Computer Vision Library,"* Available: https://opencv.org

[2] InsightFace, *"Deep Face Analysis Toolkit,"* Available: https://github.com/deepinsight/insightface

[3] Flask, *"Flask Web Framework Documentation,"* Available: https://flask.palletsprojects.com

[4] React, *"React.js Official Documentation,"* Available: https://react.dev

[5] SQLite, *"SQLite Database Documentation,"* Available: https://www.sqlite.org

[6] ONNX Runtime, *"ONNX Runtime for AI Models,"* Available: https://onnxruntime.ai

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning.* Cambridge, MA, USA: MIT Press, 2016.

[8] R. Szeliski, *Computer Vision: Algorithms and Applications.* Springer, 2010.

[9] IEEE Xplore, *"Research Papers on Face Recognition,"* Available: https://ieeexplore.ieee.org

[10] Springer, *"Journals on Artificial Intelligence and Computer Vision,"* Available: https://link.springer.com

[11] Kaggle, *"Face Recognition Datasets,"* Available: https://www.kaggle.com

[12] MDN Web Docs, *"HTML, CSS, JavaScript Documentation,"* Available: https://developer.mozilla.org

# APPENDIX

This section provides a brief overview of the important code components used in the system.

Backend (Flask API Example):

```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    name = data.get("name")
    if not name:
        return jsonify({"error": "Name required"}), 400
    return jsonify({"message": f"User {name} registered successfully"})
```

Frontend (React Example):

```
function Register() {
  const [name, setName] = useState("");

  const handleSubmit = () => {
    fetch("/register", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({ name })
    });
  };

  return (
    <input value={name} onChange={(e) => setName(e.target.value)} />
  );
}
```

Appendix B: Sample Output Screens

Face detected with bounding box .Name displayed on screen .Attendance marked with date and time

Appendix C: Tools and Technologies Used

Frontend: React.js, HTML, CSS, JavaScript .Backend: Flask (Python) .AI: OpenCV,

InsightFace .Database: SQLite .Tools: VS Code, Postman, GitHub

Appendix D: API Endpoints (Sample)

| Method | Endpoint | Description |
|---|---|---|
| POST | /register | Register new user |
| GET | /attendance | Get attendance records |
| POST | /train | Train face model |
| GET | /video_feed | Live camera streaming |

Appendix E: Dataset Information

Images captured using webcam .Stored in local dataset folder .Used for training face recognition model .Preprocessed for better accuracy

Appendix F: Limitations

Performance depends on lighting conditions .Requires camera availability .Accuracy depends on dataset quality

Appendix G: System Interface Screens

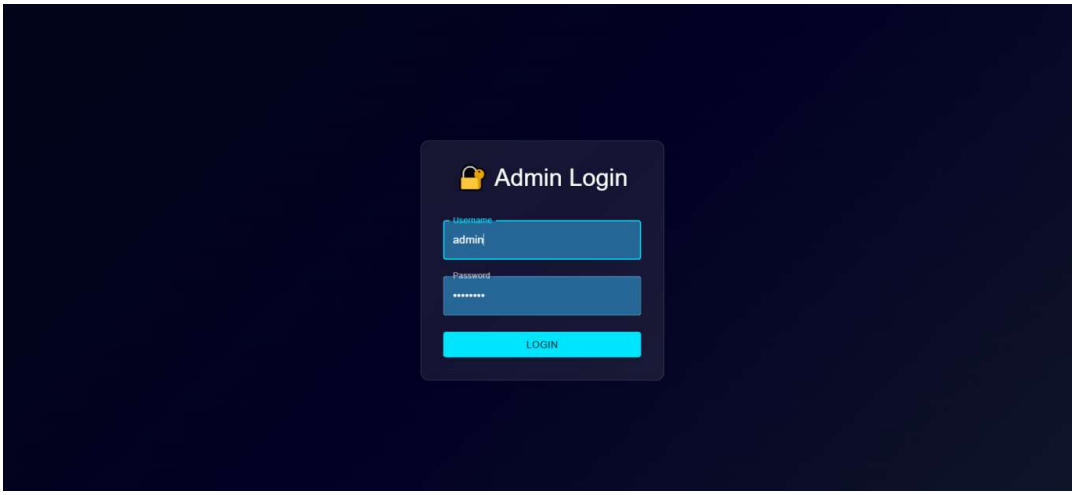



Figure 6.4.1: Admin Login Page

The Admin Login page allows authorized users to securely access the system by entering valid credentials. It ensures restricted access and protects system data from unauthorized users.

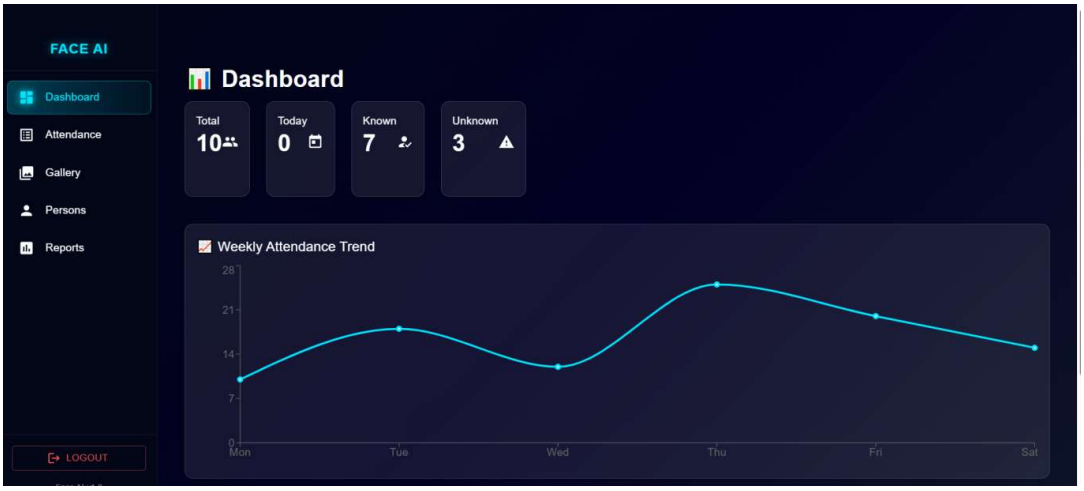



Figure 6.4.2: Dashboard

The Dashboard provides an overview of the system, displaying key information such as total users, attendance status, and quick navigation options for different modules.

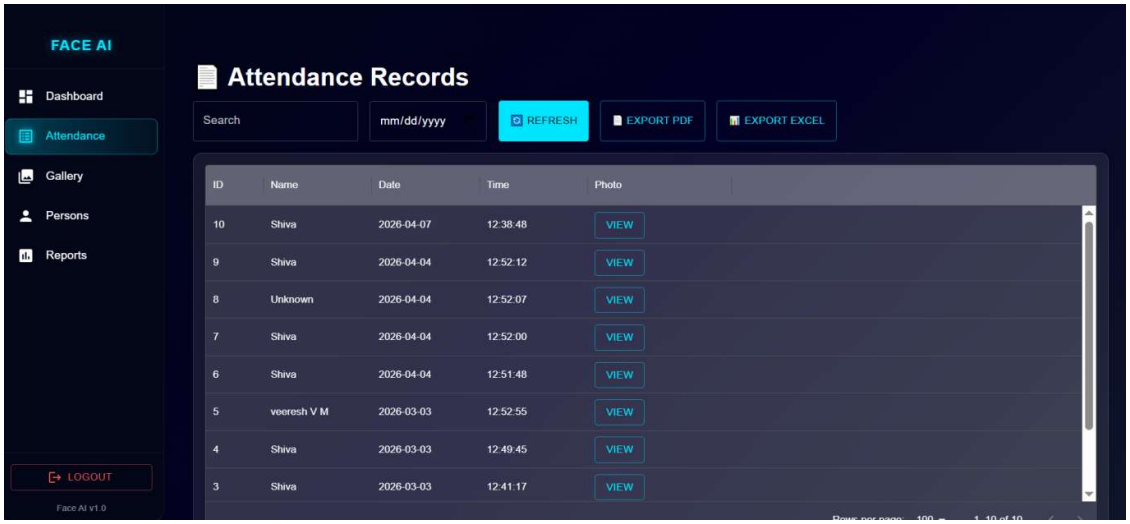



Figure 6.4.3: Attendance Page

The Attendance page displays real-time attendance records, allowing users and administrators to monitor and verify attendance details efficiently.

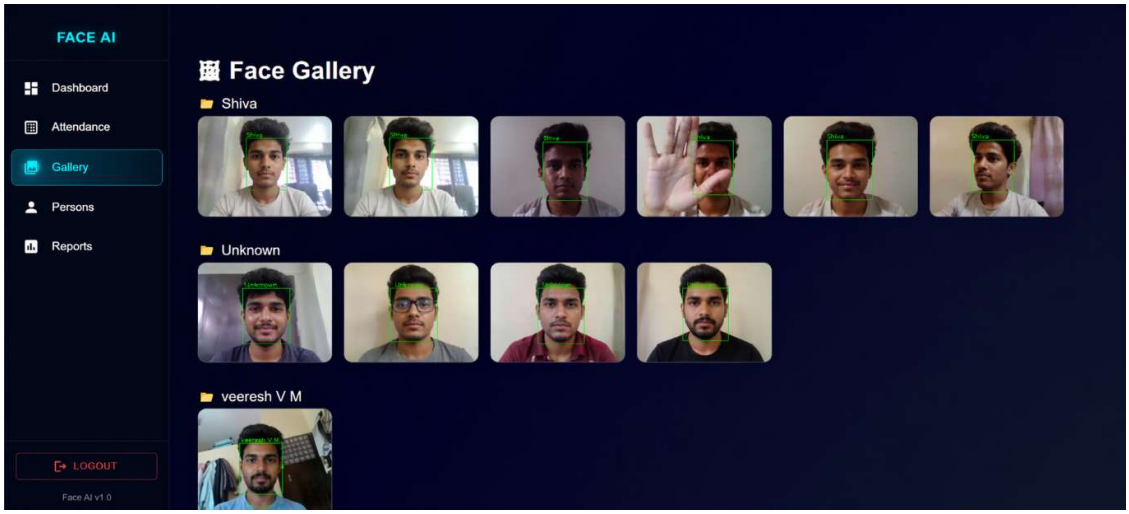



Figure 6.4.4: Gallery Page

The Gallery page shows the stored face dataset images captured during registration. It helps in verifying and managing the dataset used for training the recognition model.

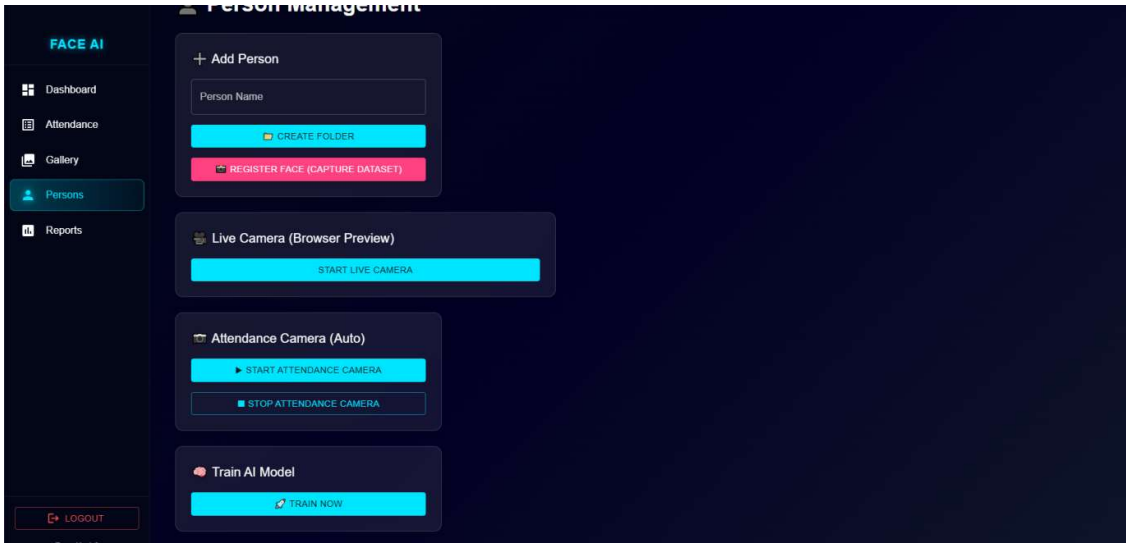



Figure 6.4.5: Person Page

The Person page contains detailed information about registered users, including personal details and associated face data used for identification.

Figure 6.4.6: Report Page

The Report page provides a structured view of attendance records, enabling analysis and tracking over time. It helps in generating insights and maintaining proper records.

**A.1 Data Sheets (AI Model & Processing Specifications)**

*(Since this is an AI-based project, this section includes model details, algorithms, and processing methods instead of hardware datasheets.)*

a. Face Recognition Algorithm Specifications

Table A.1: AI Model Specifications

| Algorithm / Model | Description | Key Parameters Used | Source |
|---|---|---|---|
| InsightFace (CNN) | Deep learning-based face recognition model using embeddings | Model weights, embedding size | InsightFace Documentation |
| OpenCV Haar Cascade | Face detection algorithm for identifying faces in images | scaleFactor, minNeighbors | OpenCV Documentation |
| Cosine Similarity | Measures similarity between face embeddings | Distance threshold | SciPy Library |
| Face Embeddings | Numerical representation of facial features | Vector size (128/512) | InsightFace |

b. Pre-processing Steps

The preprocessing of face data is performed before model training to ensure accuracy and

consistency. The process includes capturing multiple face images using a webcam, detecting faces using OpenCV, resizing and normalizing images, and generating embeddings using InsightFace. The dataset is then stored and used for real-time recognition. These preprocessing steps ensure reliable and efficient face recognition performance.

### A.2 Images of Project Output

This section includes screenshots of the developed system, demonstrating various functionalities such as:

- Admin Login Page
- Dashboard Interface
- Attendance Page
- Gallery Page
- Person Management Page
- Report Generation Page

*(Insert your screenshots here with proper figure captions like Fig A.2.1, etc.)*

### A.3 Auxiliary Documents

This section includes technical details and supporting components used during the project development:

- Python Version: 3.10
- Frontend: React.js
- Backend: Flask (Python)
- Libraries Used: OpenCV, InsightFace, NumPy, Pandas, SciPy
- Database: SQLite
- Model Storage: Face embeddings stored using Pickle

### A.4 GitHub Repository

The complete source code of the project is available at: https://github.com/Shivprasad7-hub/face-attendance.git